2026-2027 学年第一学期

《高级语言程序设计(C++语言)》

实验报告

实验名称：函数的定义、调用与调试	
实验日期：2026 年 10 月 25 日	实验地点：于刚宋晓楼
学　　号：2024301051244	姓　　名：卫金男
学　　院：	专　　业：人工智能专业
指导老师：	

一、实验目的

掌握函数的声明、定义和调用，比较值参数与引用参数，练习函数重载、默认参数和内联函数。

完成独立控制台程序，使用 Visual Studio 的断点、逐语句和逐过程功能观察形参、局部变量与递归调用。

练习使用标准库函数，例如用 std::isfinite 检查浮点数是否为有限值。

二、实验环境

实验在本地 Windows 上完成，使用 Visual Studio Community 2026，编译器为 MSVC 19.51。

每题建立独立的 C++ 控制台项目，采用 Debug x64 配置和 C++17 标准。

运行结果通过标准输出重定向保存到 output.txt，再在 Visual Studio 中打开文件截图。

三、实验过程及结果

题目 1　课堂练习：阶乘函数

将阶乘计算写为独立函数，求 5!；比较单步进入与单步跳过，观察形参的变化和作用域，并分别检查四种调用。

main 调用 fact(5)。fact 将累积值设为 1，每轮先乘当前 val，再递减 val，最后返回乘积。

解答：代码

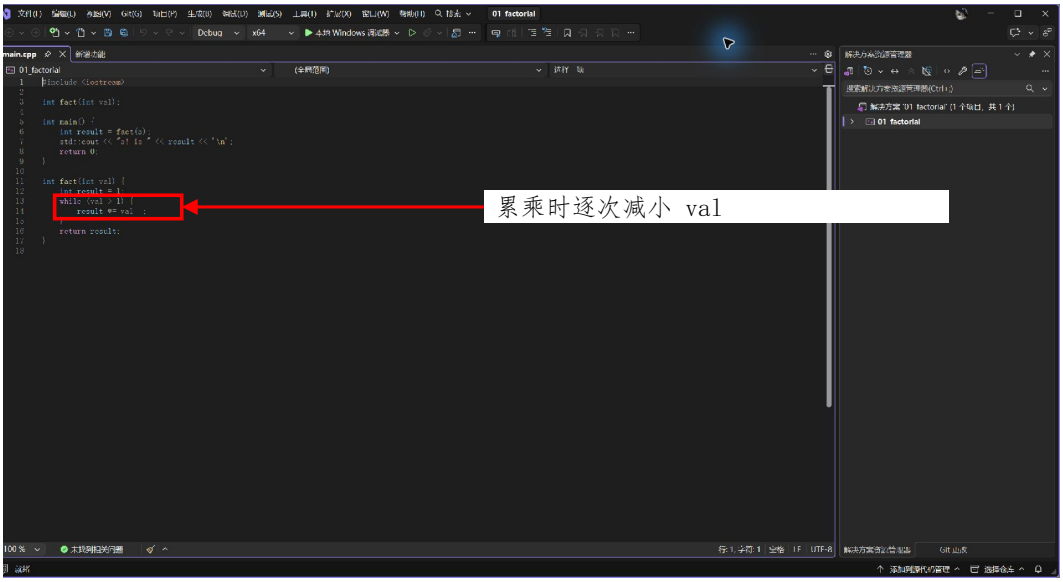


图 1 阶乘程序：main 调用与 fact 累乘

main 调用 fact(5) 并接收返回值。fact 从 1 开始累乘，同时将形参 val 逐次减小，最后返回阶乘结果。

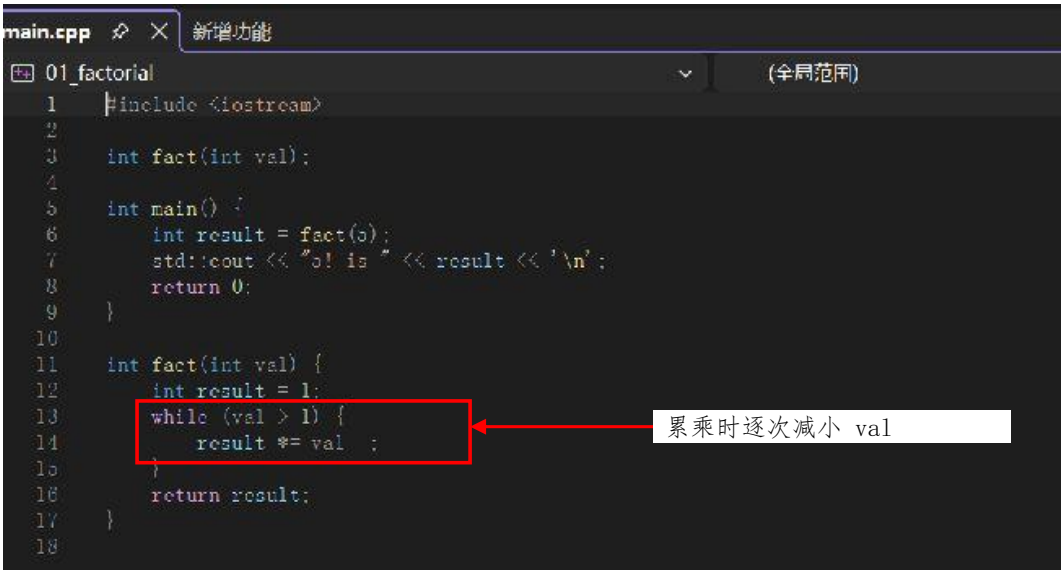


图 2 阶乘程序：main 调用与 fact 累乘

main 调用 fact(5) 并接收返回值。fact 从 1 开始累乘，同时将形参 val 逐次减小，最后返回阶乘结果。

运行结果与调试

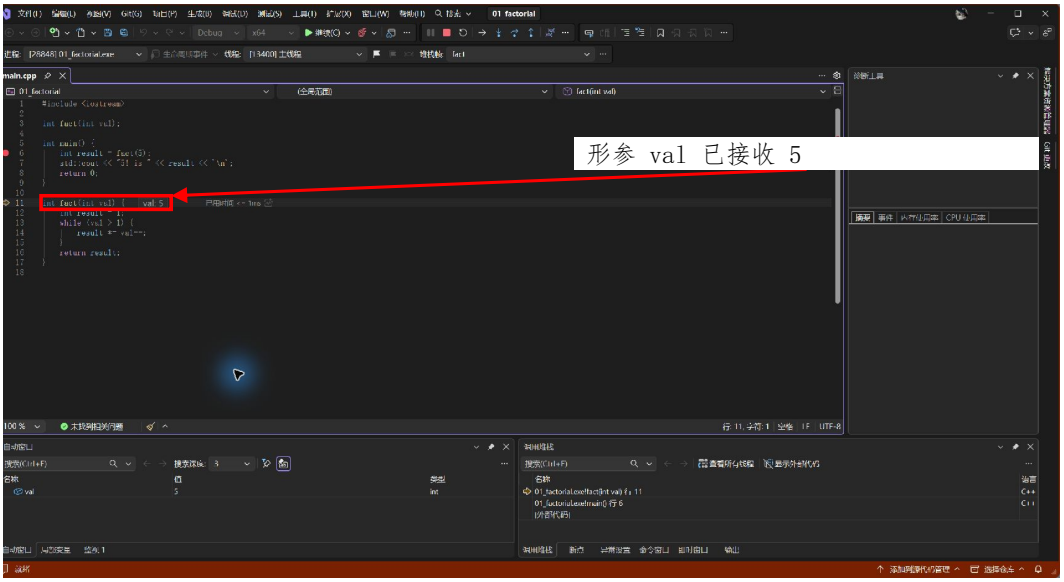


图 3　逐语句进入 fact 的停靠位置

逐语句进入 fact 后，当前停在函数起始处，形参 val=5。

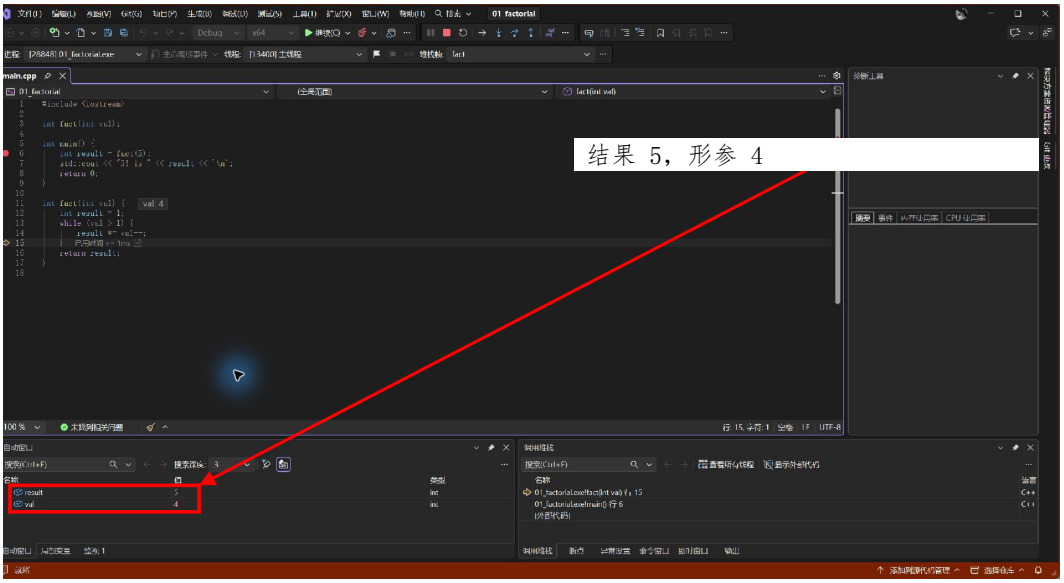


图 4　一次累乘后的 result 与 val

执行第一次乘法后，result=5、val=4。

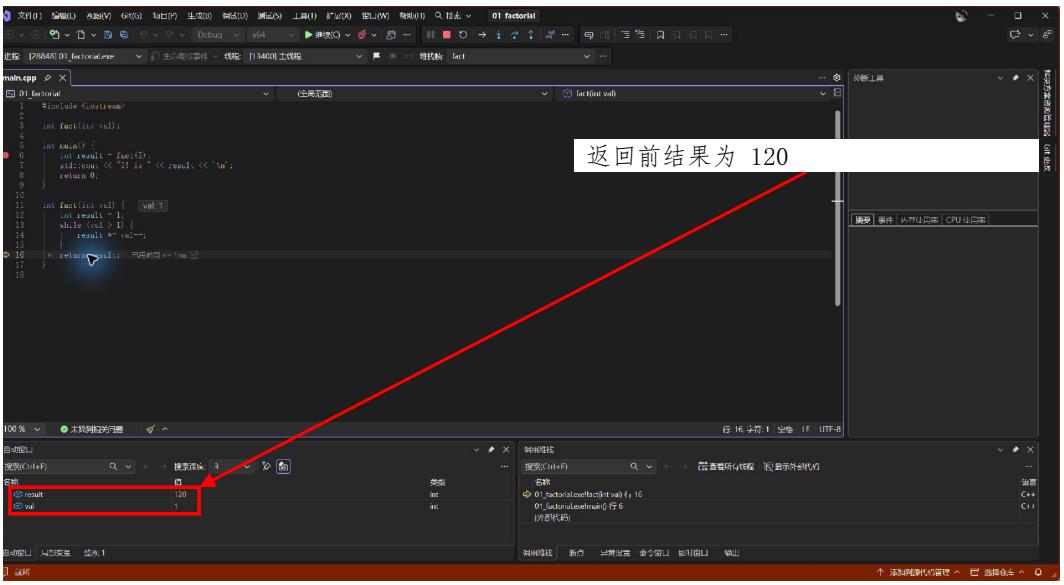


图 5 返回前的 result 与 val

循环结束时，result=120、val=1，下一步将返回 result。

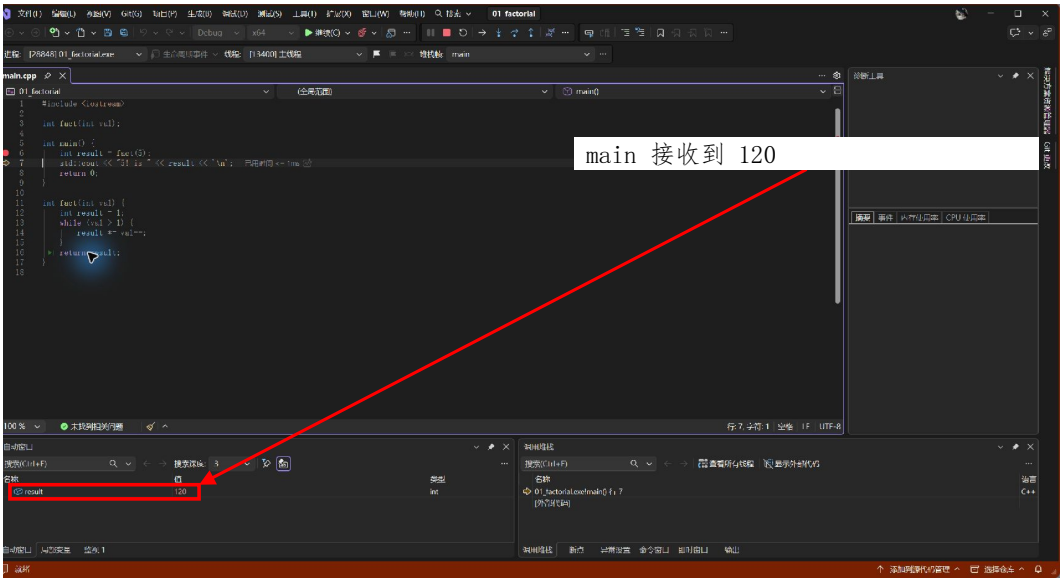


图 6 main 接收到返回值 120

回到 main 后，main 中的 result 已接收到 120。

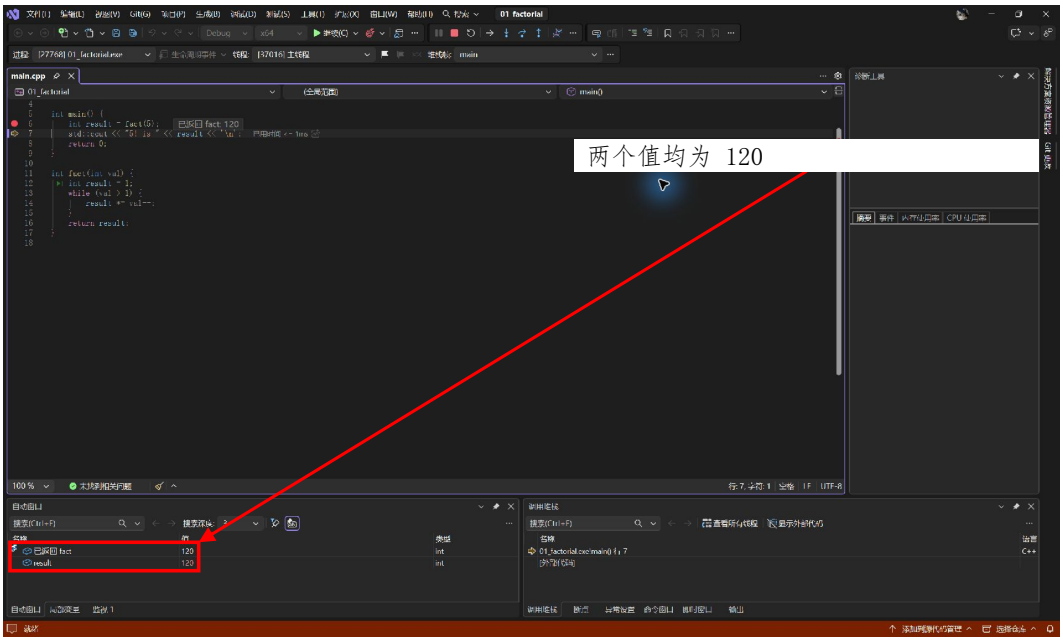


图 7　逐过程后的返回值与 result

逐过程完成调用后，返回值和 main 中的 result 均为 120。

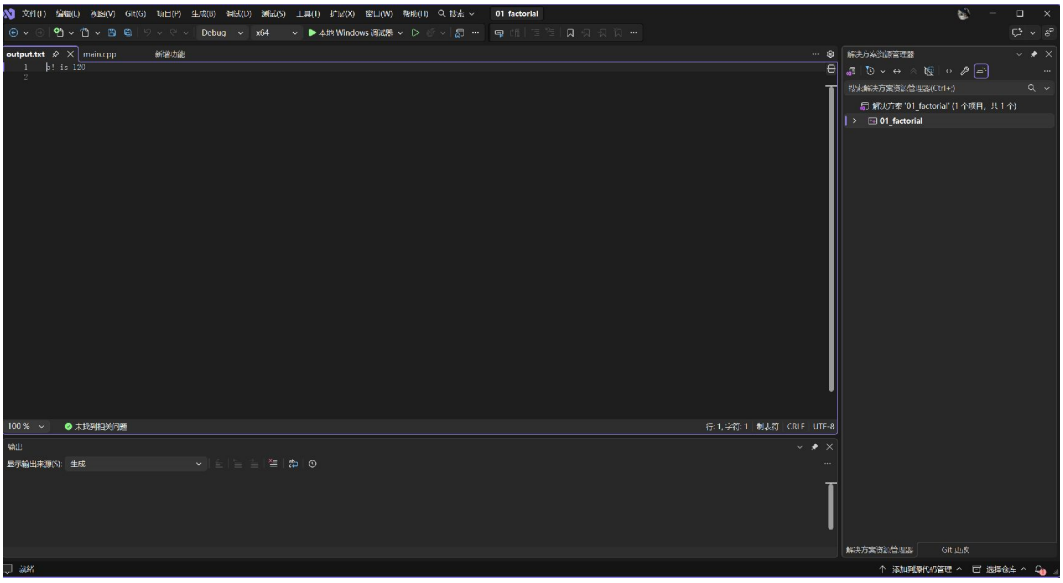


图 8　VS 运行生成的标准输出文件 output.txt

在 Visual Studio 中实际运行程序后，output.txt 中显示 5! is 120。这里展示的是标准输出重定向生成的文件内容。

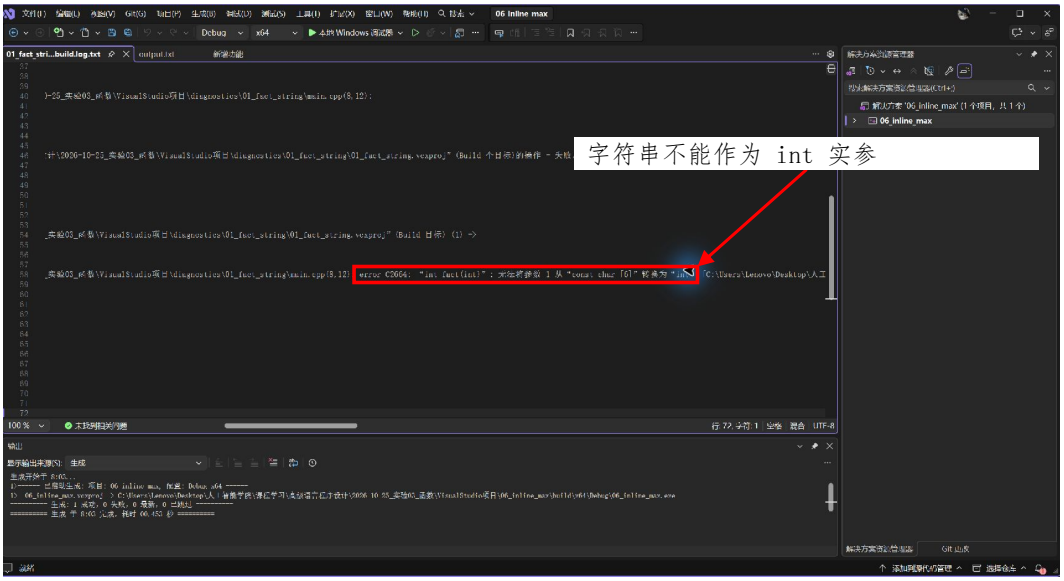


图 9 历史 MSVC 命令行构建日志：fact("hello") 的 C2664

字符串字面量不能转换为 fact 的 int 形参，因此构建失败。

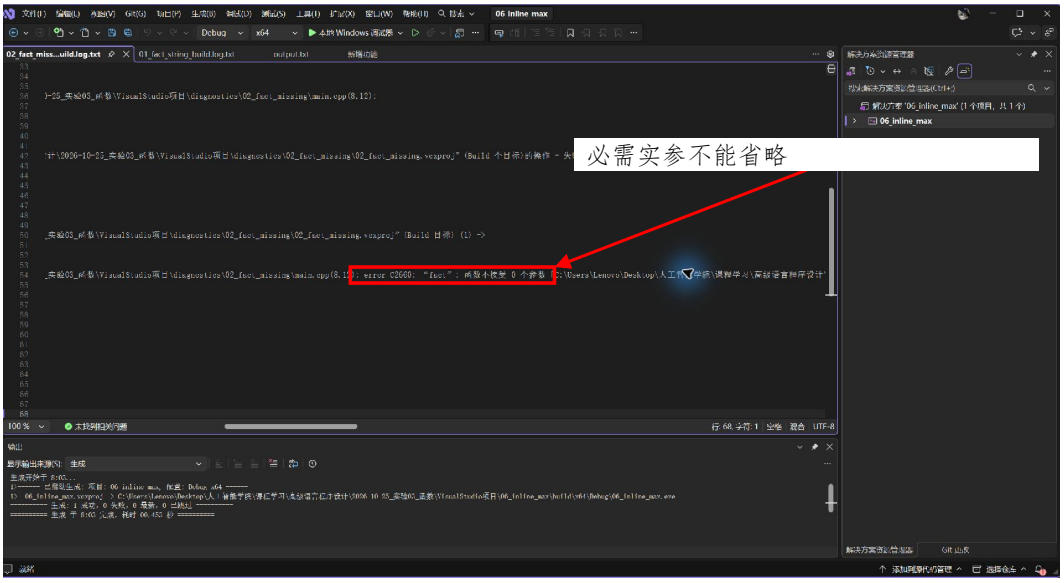


图 10 历史 MSVC 命令行构建日志：fact() 的 C2660

fact 需要一个实参，fact() 没有提供必需实参，日志报告 C2660。

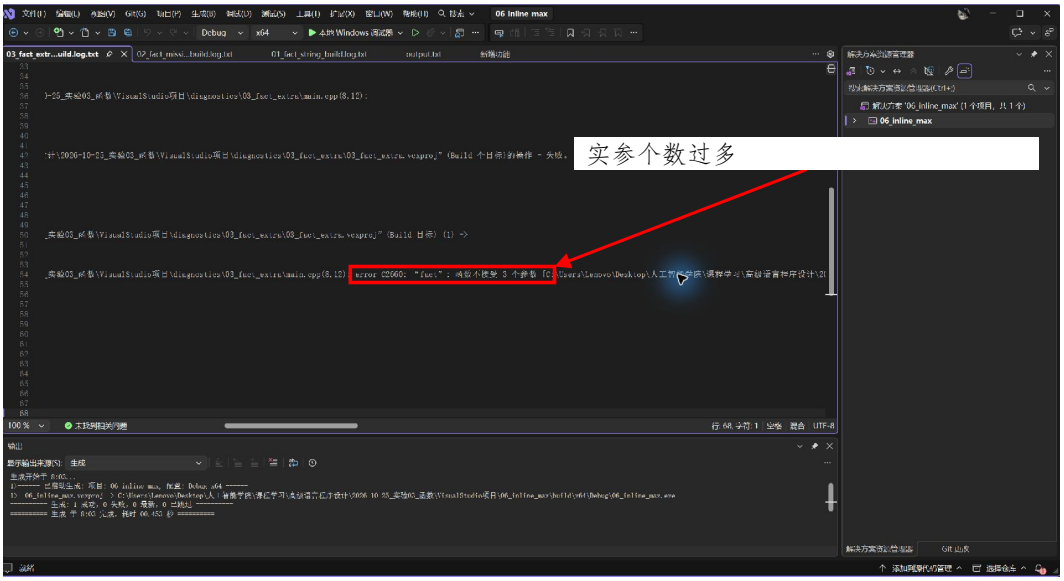


图 11 历史 MSVC 命令行构建日志：fact(42,10,0) 的 C2660

fact 只接收一个实参，传入三个实参的调用使日志报告 C2660。

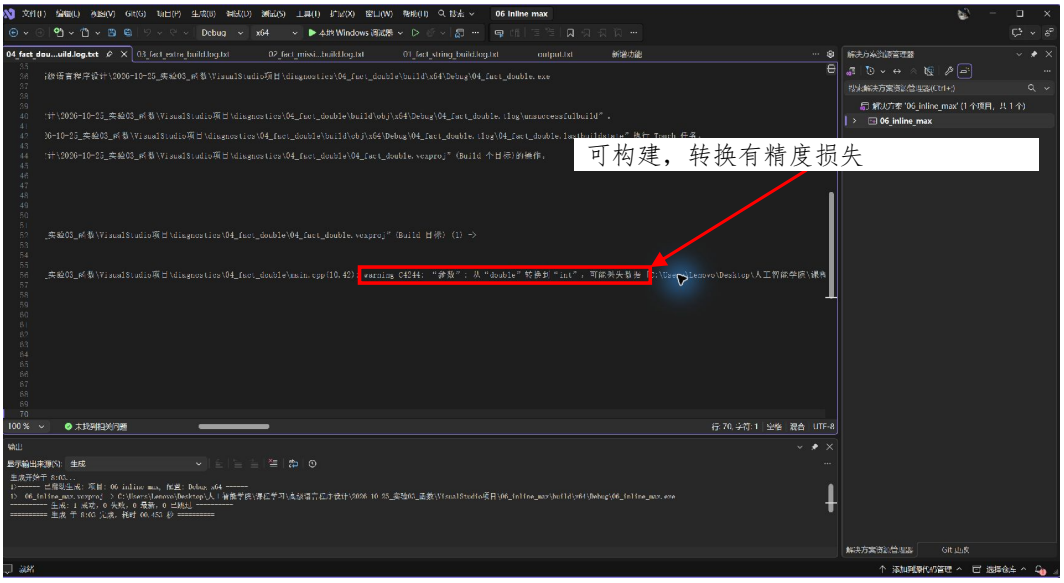


图 12 历史 MSVC 命令行构建日志：fact(3.14) 的 C4244

double 转为 int 可能丢失小数部分，编译器给出 C4244 警告；该项目构建成功。

在 main 第 6 行按 F11 进入 fact，形参 val 为 5；第一次累乘后，result=5、val=4。

返回语句前，result=120、val=1；回到 main 第 7 行，main 中的 result 为 120。

在同一调用行按 F10 逐过程，直接到 main 第 7 行，返回值和 result 均为 120。val 随该次 fact 调用结束而离开作用域，两个函数中的 result 各自独立。

四个调用变式的 MSVC 构建记录中，字符串实参报 C2664，缺少或多传实参报 C2660；实数实参产生 C4244 警告后可正常生成，fact(3.14)的运行结果为 6。

题目 2　课堂练习：引用参数交换

在 main 中输入两个整数，依次调用 swap(a,a) 和 swap(a,b)，单步观察两个调用的执行路径。

两个形参都是 int 引用。数值相等时直接返回，否则先用 temp 保存 v2，再完成两次赋值。

解答：代码

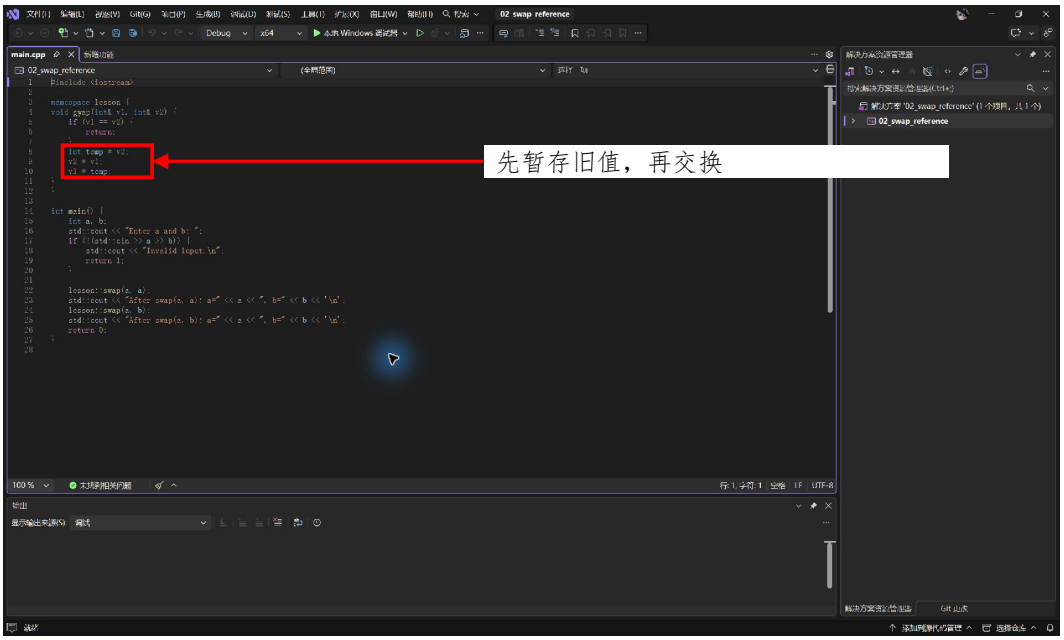


图 13　引用交换函数与提前返回条件

引用形参直接对应调用方的变量。两个值相等时直接返回，否则先用 temp 保存旧值，再完成两次赋值。

main 读入两个整数后，先调用 swap(a,a)，再调用 swap(a,b)。两次输出用于比较同一变量与两个变量作实参时的结果。

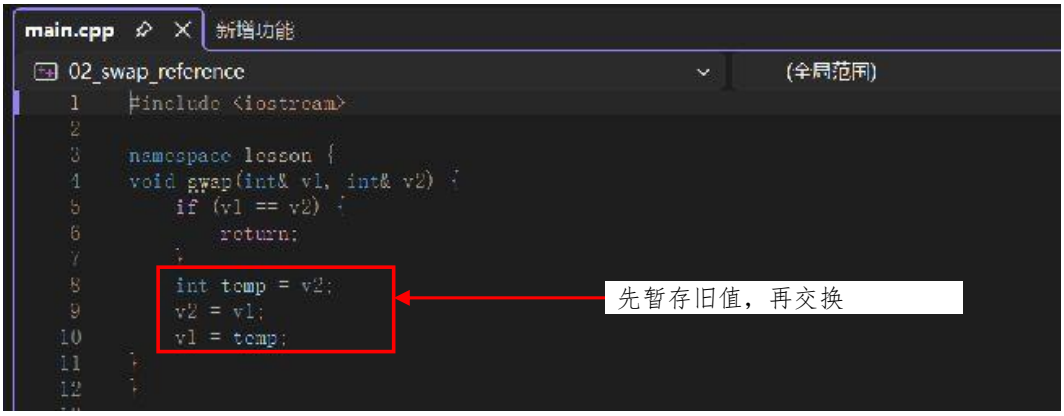


图 14 引用交换函数与提前返回条件

引用形参直接对应调用方的变量。两个值相等时直接返回，否则先用 temp 保存旧值，再完成两次赋值。

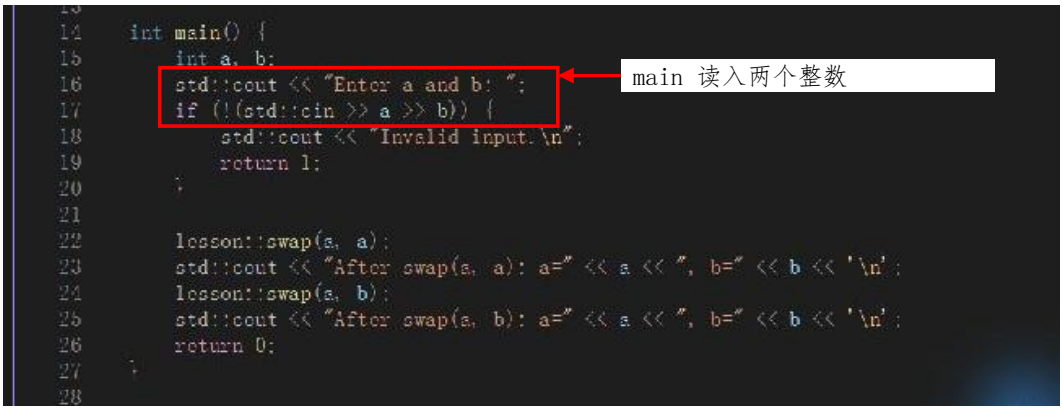


图 15 引用交换的输入与两次函数调用

main 读入两个整数后，先调用 swap(a,a)，再调用 swap(a,b)。两次输出用于比较同一变量与两个变量作实参时的结果。

运行结果与调试

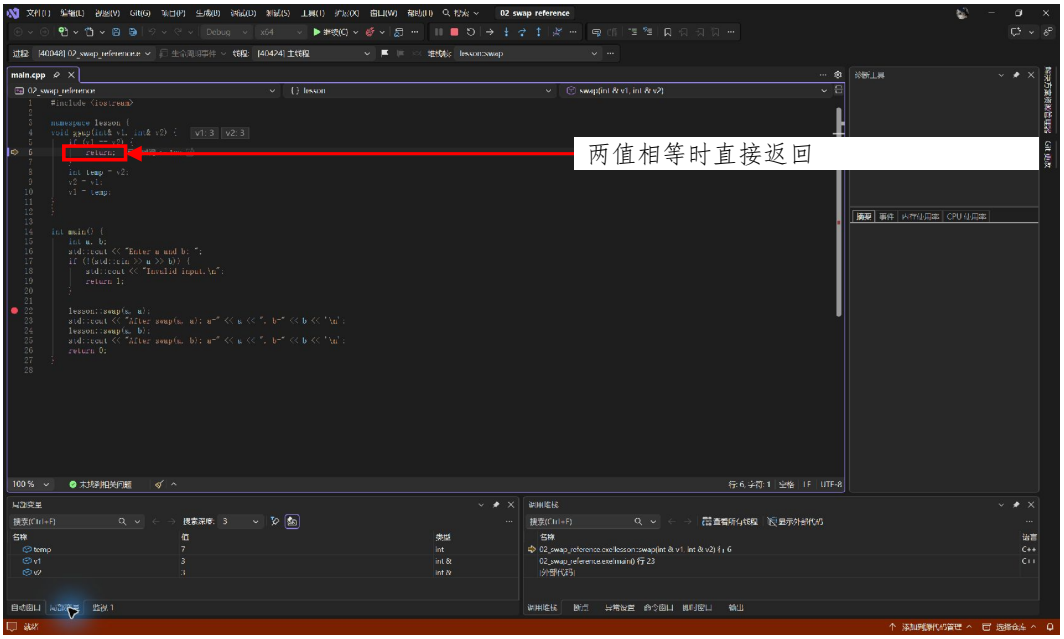


图 16　swap(a,a) 在判断后提前返回

swap(a,a)中 v1、v2 均为 3，条件成立后直接返回。

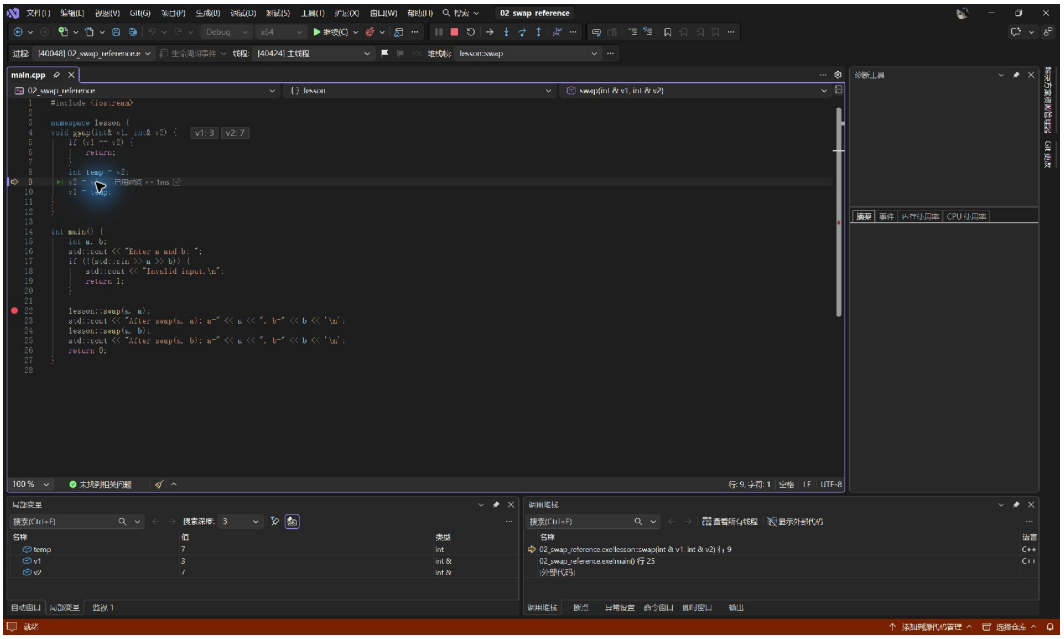


图 17　temp 保存旧值后的局部变量

temp 已保存原 v2 的值 7，此时 v1=3、v2=7。下一步才执行 v2=v1。

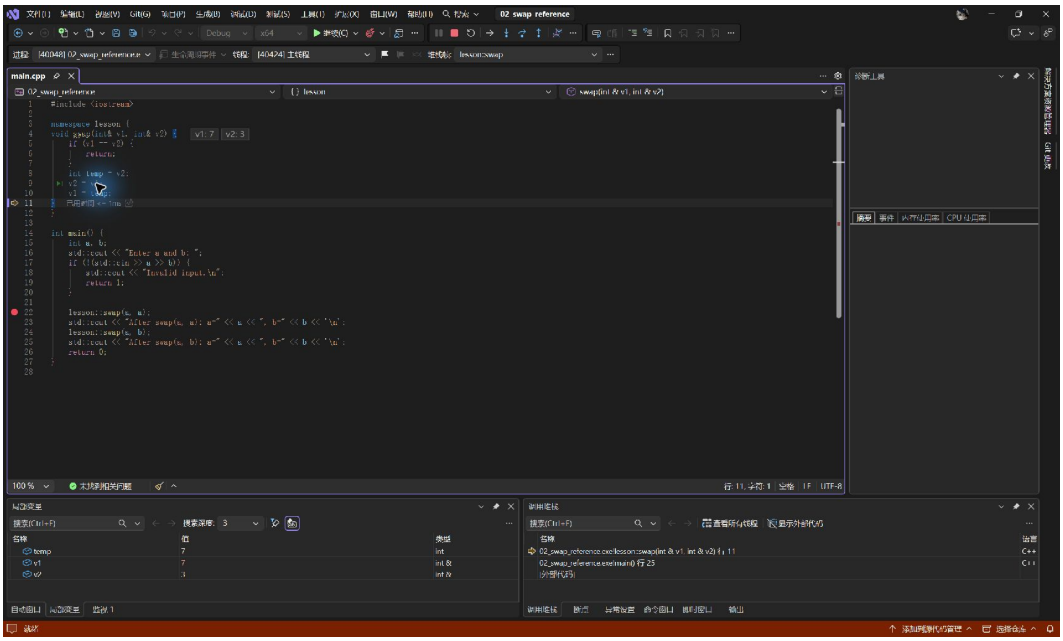


图 18　两次赋值完成后的局部变量

交换完成时 v1=7、v2=3，temp 仍为 7。引用形参的改变将反映到 main 中对应的变量。

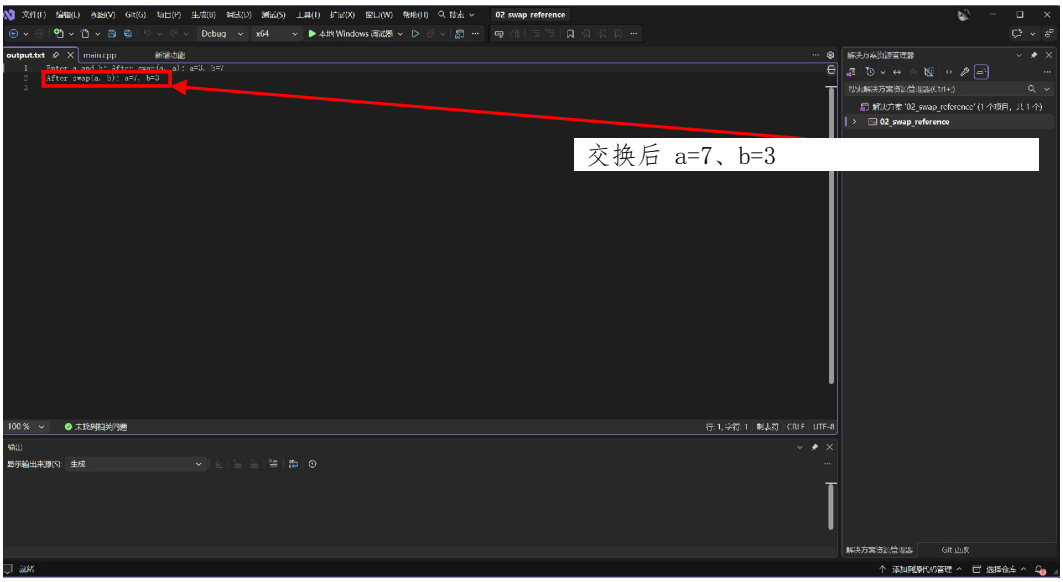


图 19　VS 运行生成的 swap 标准输出文件

输入 3 和 7 时，同一变量调用后仍为 3、7；交换两个变量后变为 7、3。

输入 3 和 7，swap(a,a)判断成立后直接返回，a、b 保持 3、7。

swap(a,b)先将 7 存入 temp，再完成两次赋值，最终 a=7、b=3。

题目 3　课堂练习：按参数类型重载

分别定义接收 int、double、long 的同名绝对值函数，观察三次调用选择的函数。

三个 lesson::abs 重载的参数类型不同。main 中的 x1、x2、x3 分别使用 int、double 和 long。

解答：代码

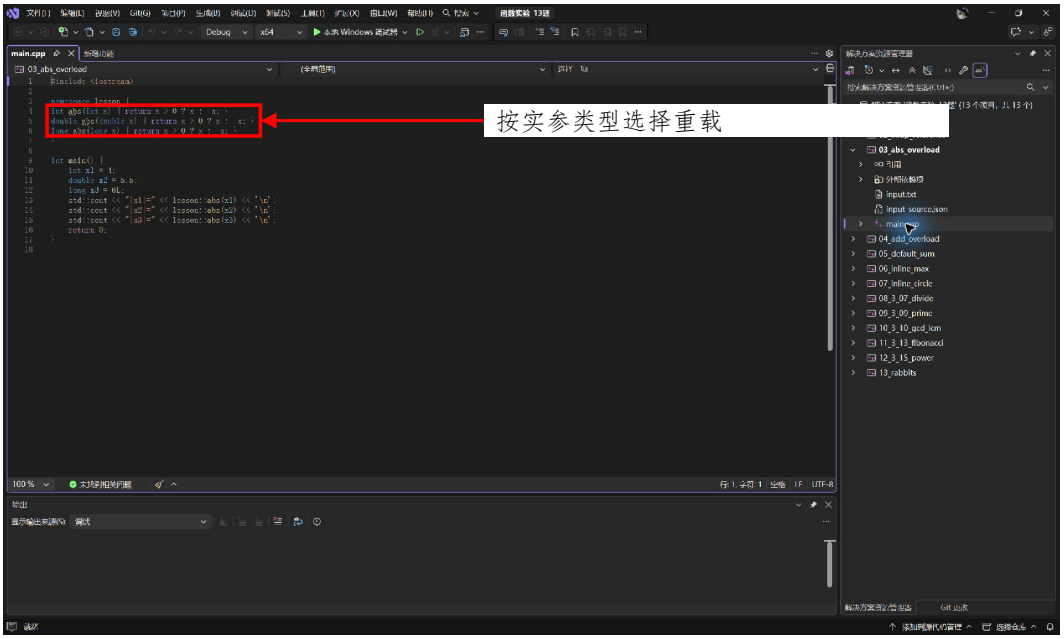


图 20　三种参数类型的 abs 重载

三个 abs 分别接收 int、double 和 long，函数体用条件表达式计算绝对值。main 用三个不同类型的变量调用同名函数，实参类型决定选中的重载。

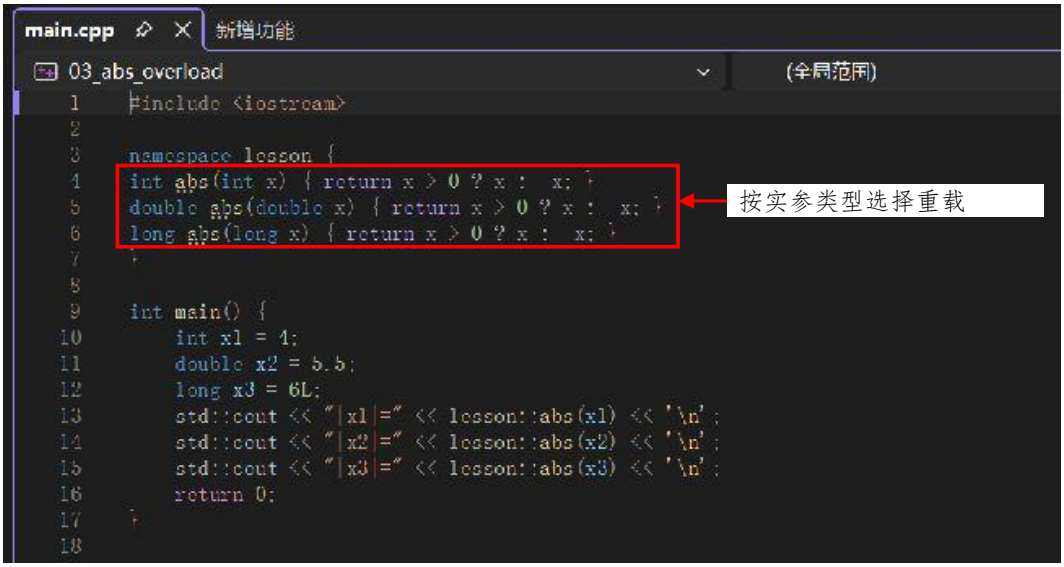


图 21　三种参数类型的 abs 重载

三个 abs 分别接收 int、double 和 long，函数体用条件表达式计算绝对值。main 用三个不同类型的变量调用同名函数，实参类型决定选中的重载。

运行结果与调试

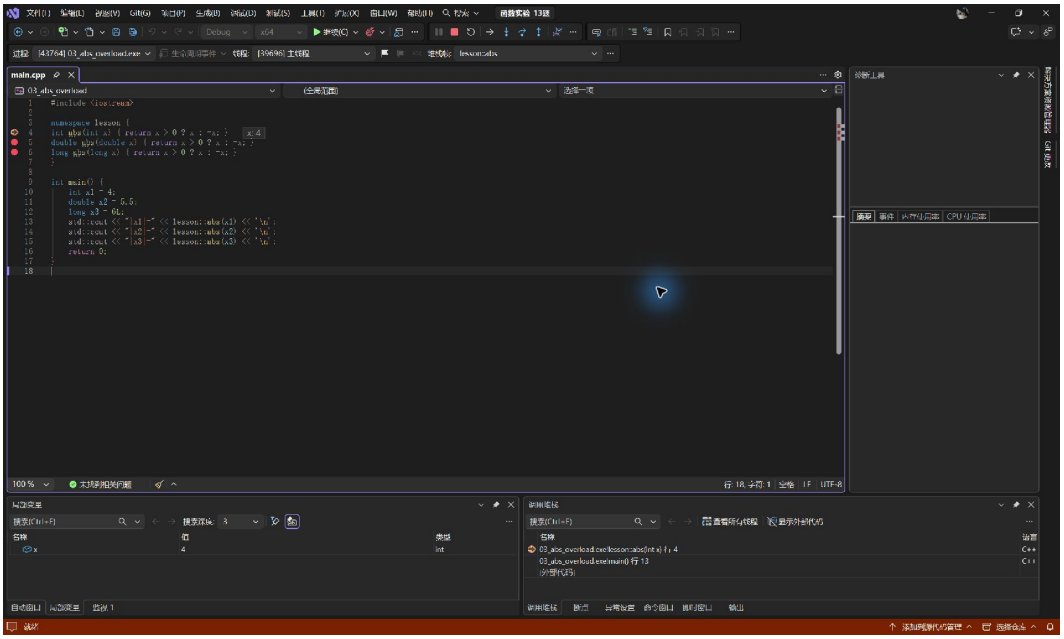


图 22 int 重载形参 x 的值与类型列

int 版本的形参 x=4，值和类型列对应 int。

当前帧为 abs(int x)，调用者是 main。

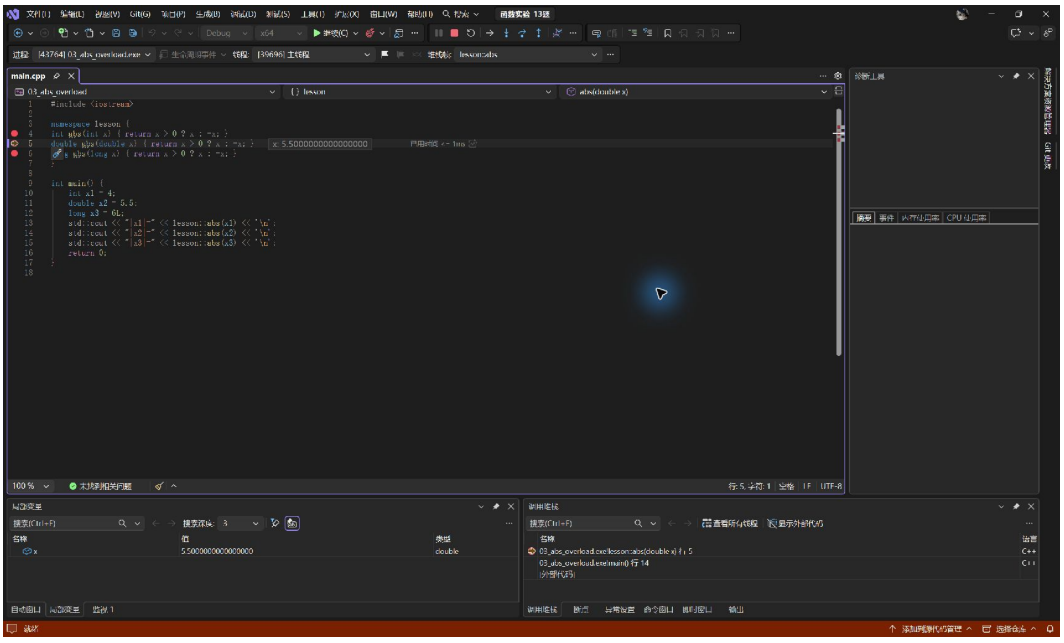


图 23 double 重载形参 x 的值与类型列

double 版本的形参 x=5.5，调试器显示其完整小数形式和 double 类型。

当前帧为 abs(double x)，调用者是 main。

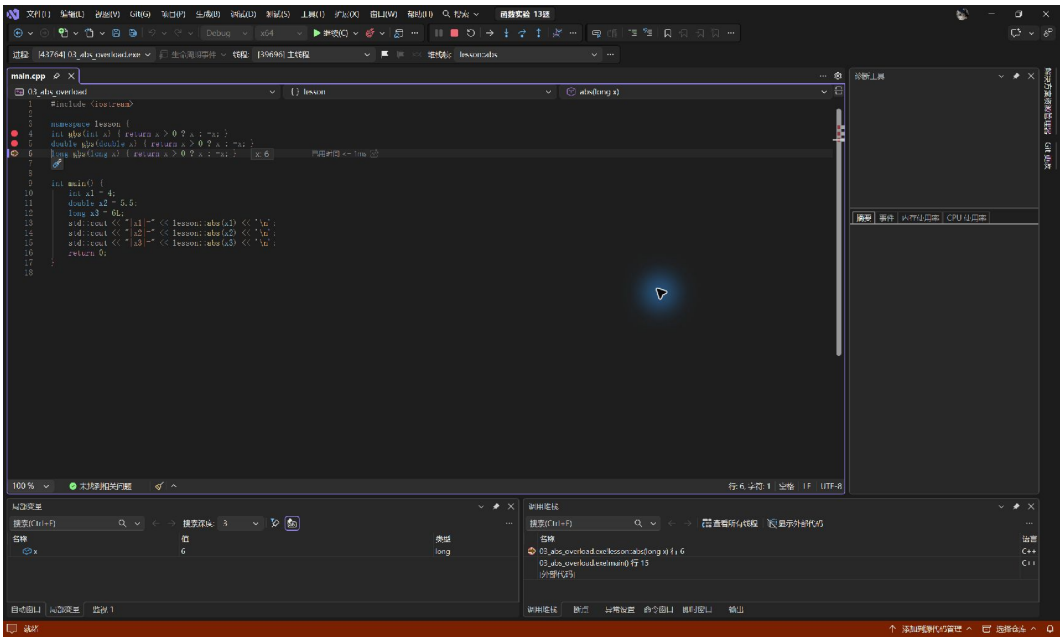


图 24　long 重载形参 x 的值与类型列

long 版本的形参 x=6，与本次 long 重载的调用堆栈对应。

当前帧为 abs(long x)，调用者是 main。

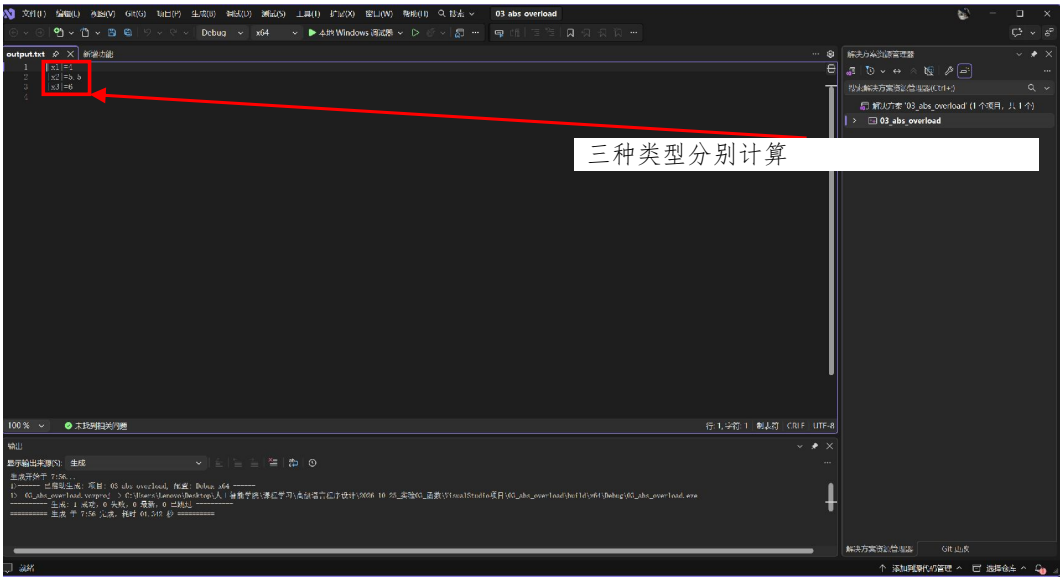


图 25　VS 运行生成的 abs 标准输出文件

三个实际输出依次为 4、5.5、6，与三种形参类型的调用对应。

三个断点依次进入 abs(int x)、abs(double x)、abs(long x)，对应形参值为 4、5.5、6。

重载由函数名和实参列表确定。若形参列表相同而仅改变返回类型，同一次调用就无法区分目标，因此不能仅靠返回类型构成重载。

题目 4　课堂练习：按参数个数重载

定义两参数和三参数两个 Add 重载，观察两次调用进入的函数。

Add(int,int) 返回两数之和,Add(int,int,int) 返回三数之和,main 根据实参数量调用对应版本。

解答：代码

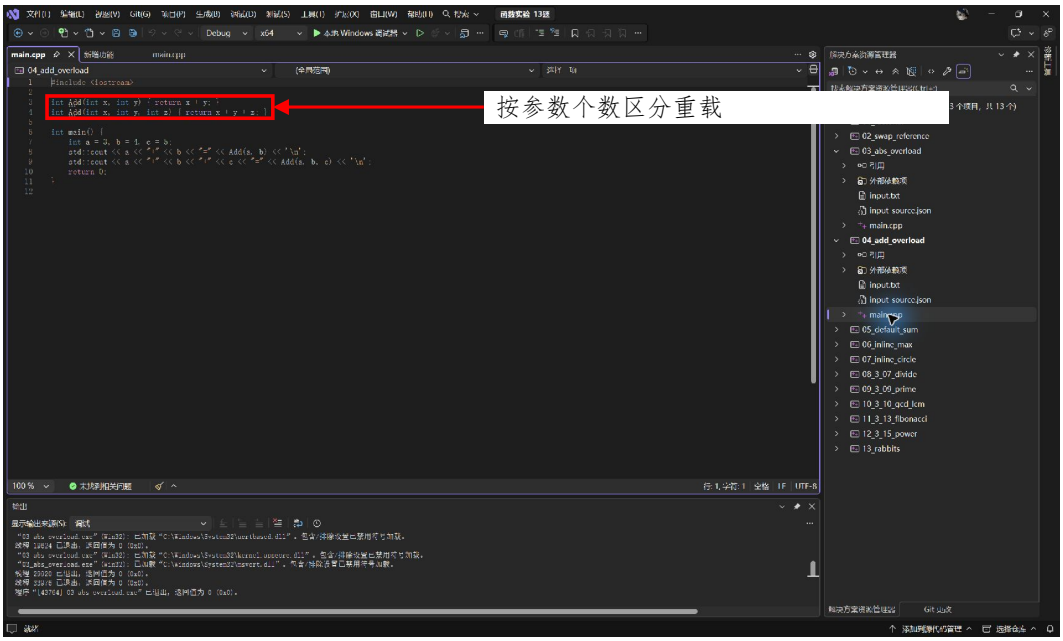


图 26 按参数个数区分的两个 Add 重载

两参数版本计算 x+y，三参数版本计算 x+y+z。main 用相同函数名分别传入两个和三个实参，输出 7 与 12。

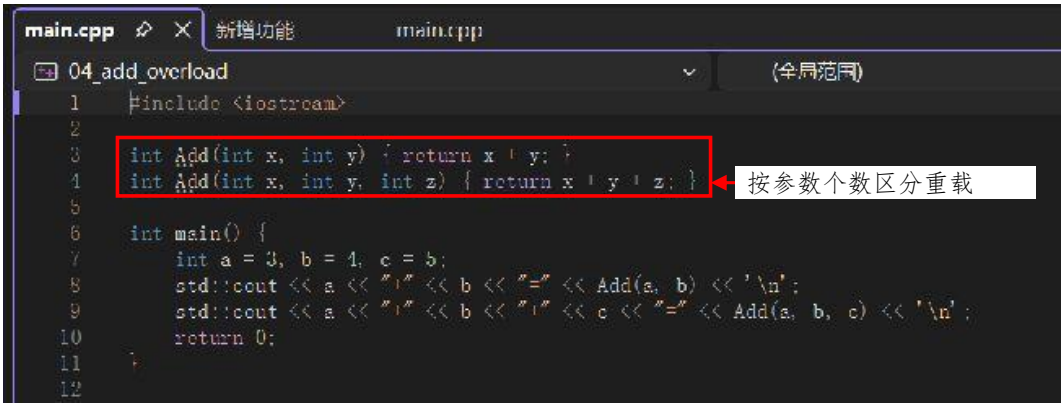


图 27 按参数个数区分的两个 Add 重载

两参数版本计算 x+y，三参数版本计算 x+y+z。main 用相同函数名分别传入两个和三个实参，输出 7 与 12。

运行结果与调试

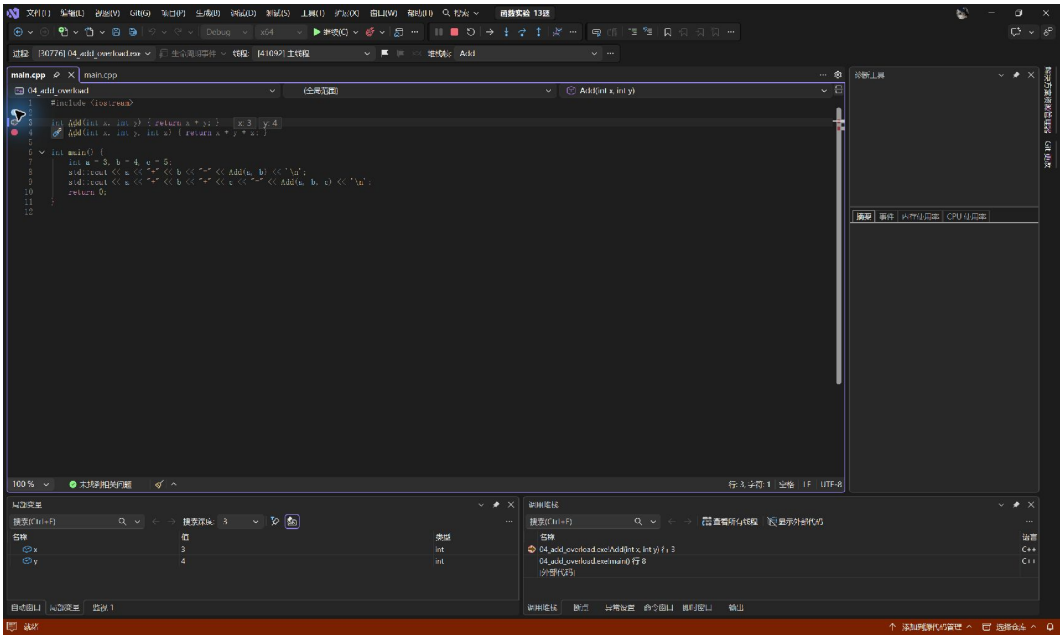


图 28　两参数 Add 的局部变量

两参数版本中的 x=3、y=4。

调用堆栈显示进入 Add(int x,int y)，由 main 调用。

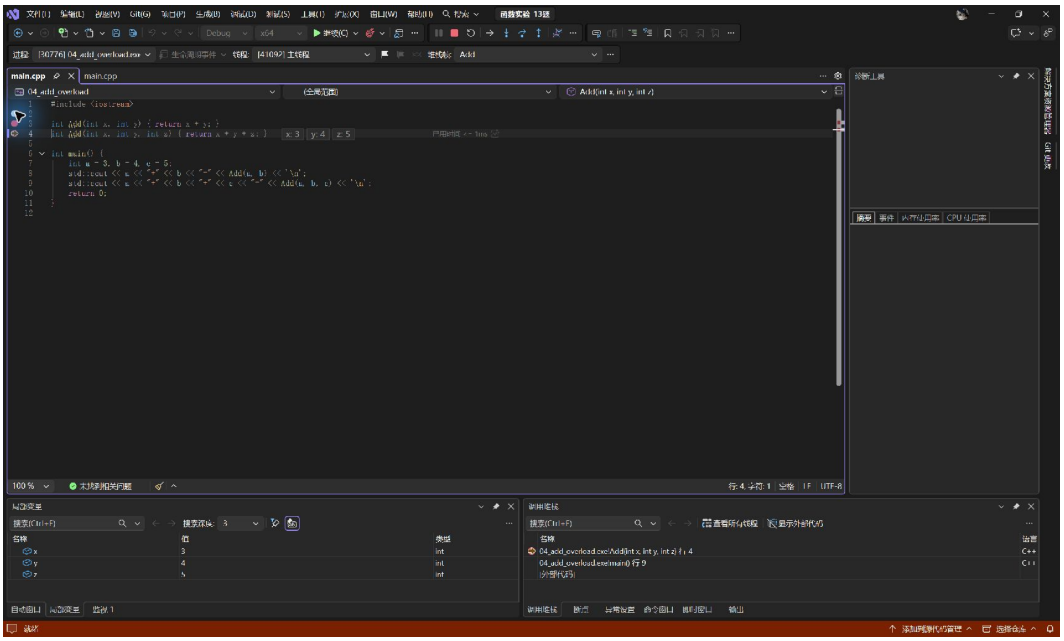


图 29　三参数 Add 的局部变量

三参数版本中的 x=3、y=4、z=5。

调用堆栈显示进入 Add(int x,int y,int z)，由 main 调用。

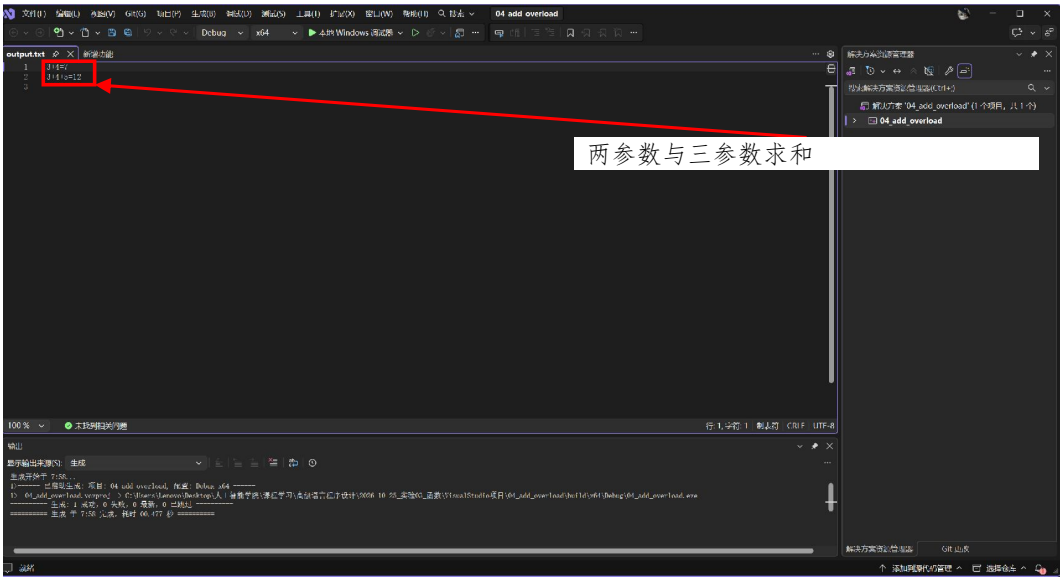


图 30 VS 运行生成的 Add 标准输出文件

两参数调用输出 3+4=7，三参数调用输出 3+4+5=12。

两参数调用进入 Add(int x,int y)，局部形参为 3、4，输出 7。

三参数调用进入 Add(int x,int y,int z)，局部形参为 3、4、5，输出 12。

题目 5 课堂练习：默认参数

调用 sum(100) 和 sum()，观察 num 的取值，并判断默认参数声明和省略实参的两个反例。

sum 的默认参数为 10，循环累加 1 至 num；提供实参时使用实参值，省略时使用调用点可见的默认值。

解答：代码

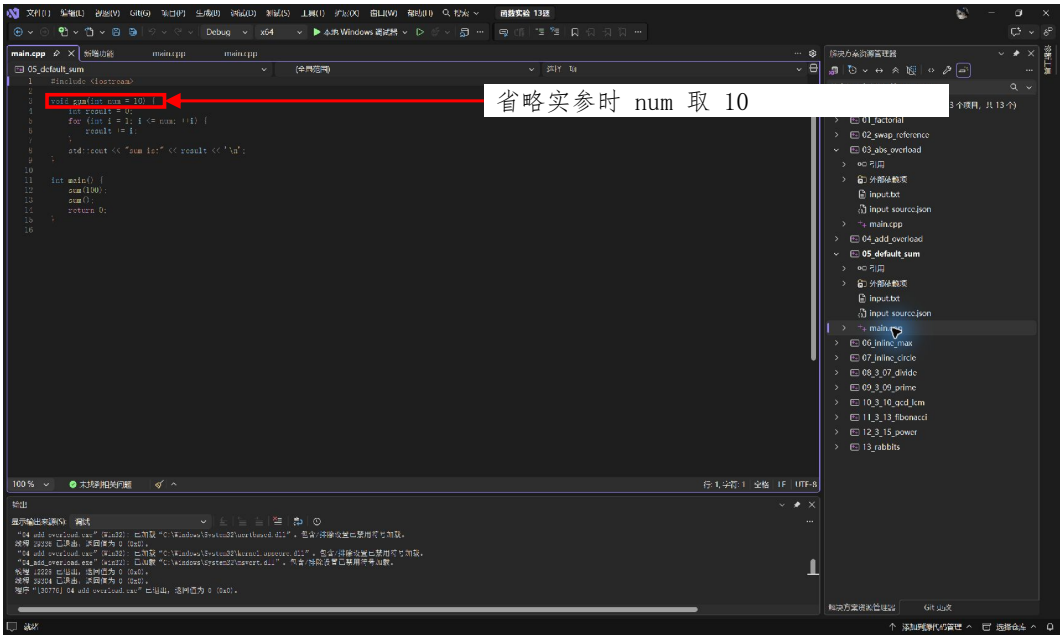


图 31 默认参数求和函数及两次调用

sum 默认将 num 设为 10，并用循环累加 1 到 num。sum(100) 使用显式实参，sum() 使用默认实参，分别得到 5050 和 55。

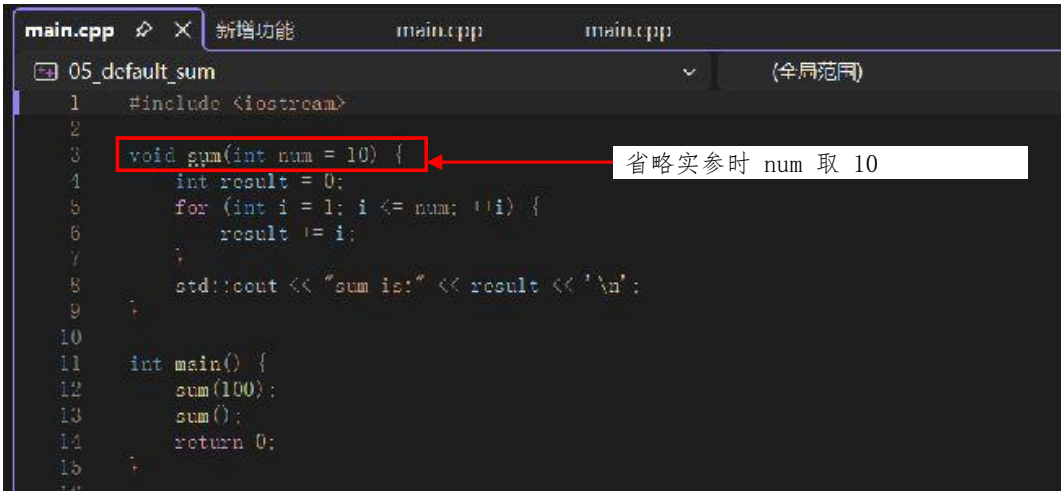


图 32 默认参数求和函数及两次调用

sum 默认将 num 设为 10，并用循环累加 1 到 num。sum(100) 使用显式实参，sum() 使用默认实参，分别得到 5050 和 55。

运行结果与调试

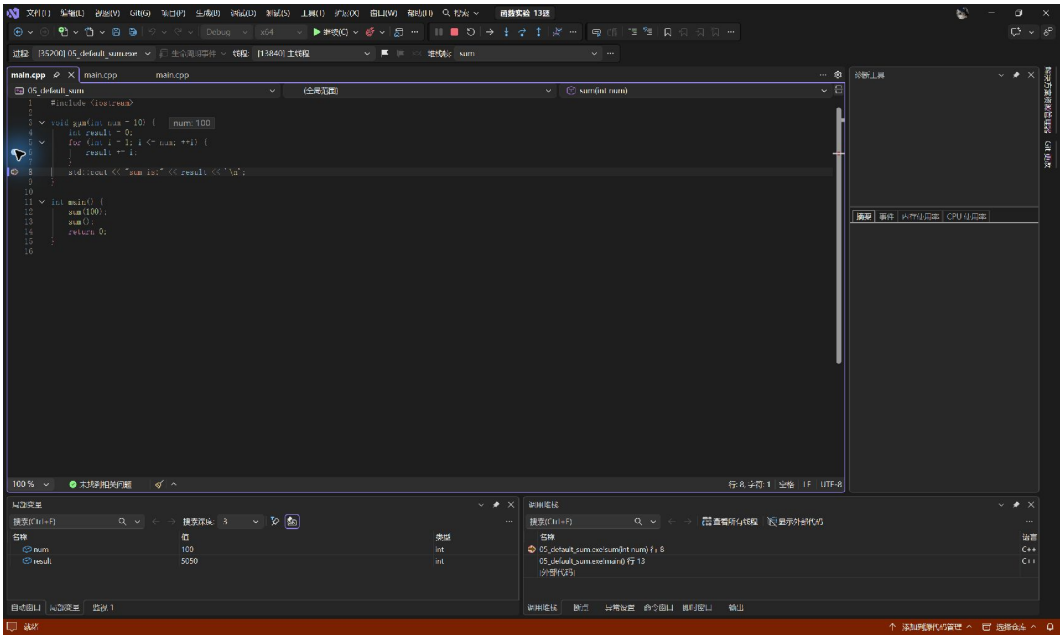


图 33 显式实参求和后的 num 与 result

显式传入 100 时，num=100，累加结果为 5050。

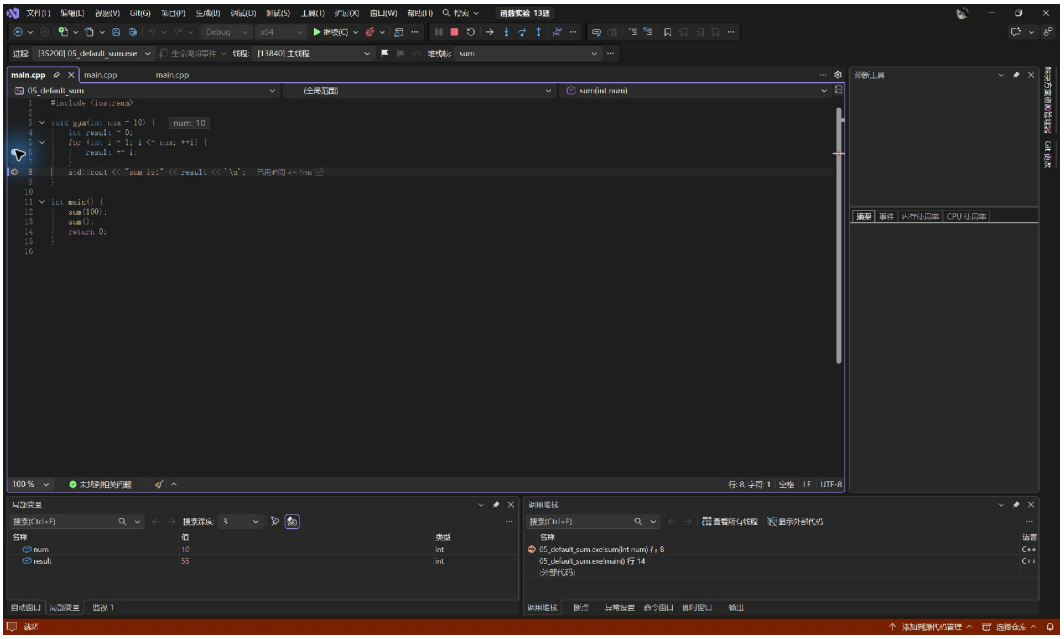


图 34 默认实参求和后的 num 与 result

省略实参时，num=10，累加结果为 55。

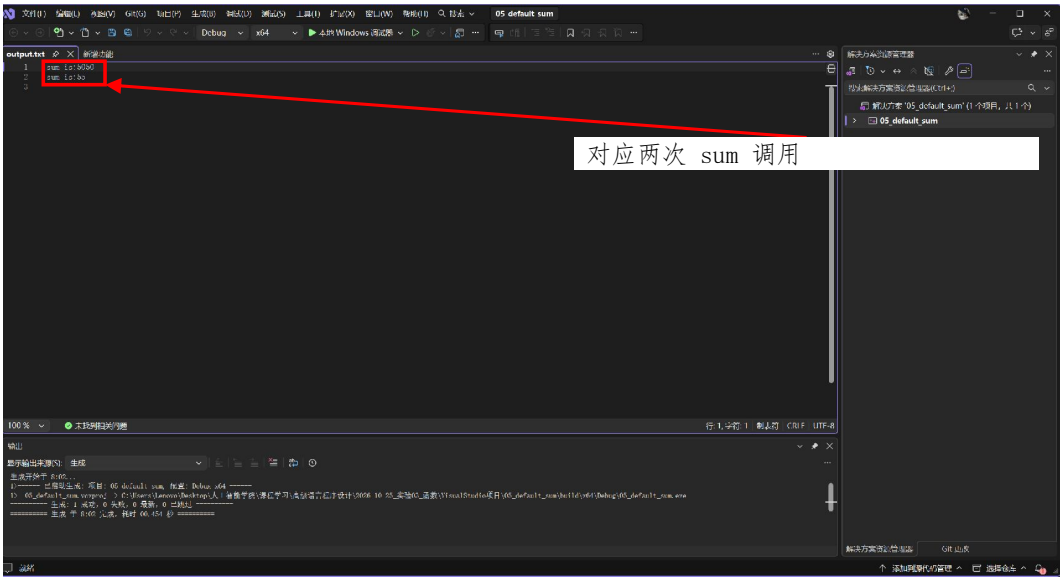


图 35 VS 运行生成的 sum 标准输出文件

sum(100) 输出 5050，省略实参的 sum() 输出 55。

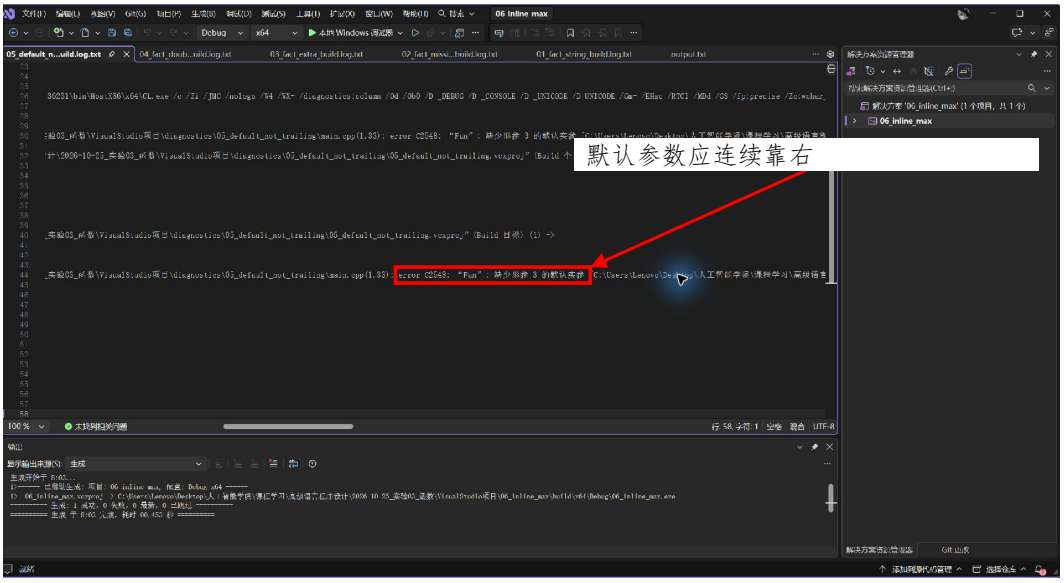


图 36 历史 MSVC 命令行构建日志：默认参数顺序的 C2548

j 已有默认值，后面的 k 也应有默认值，因此 Fun 的声明产生 C2548。

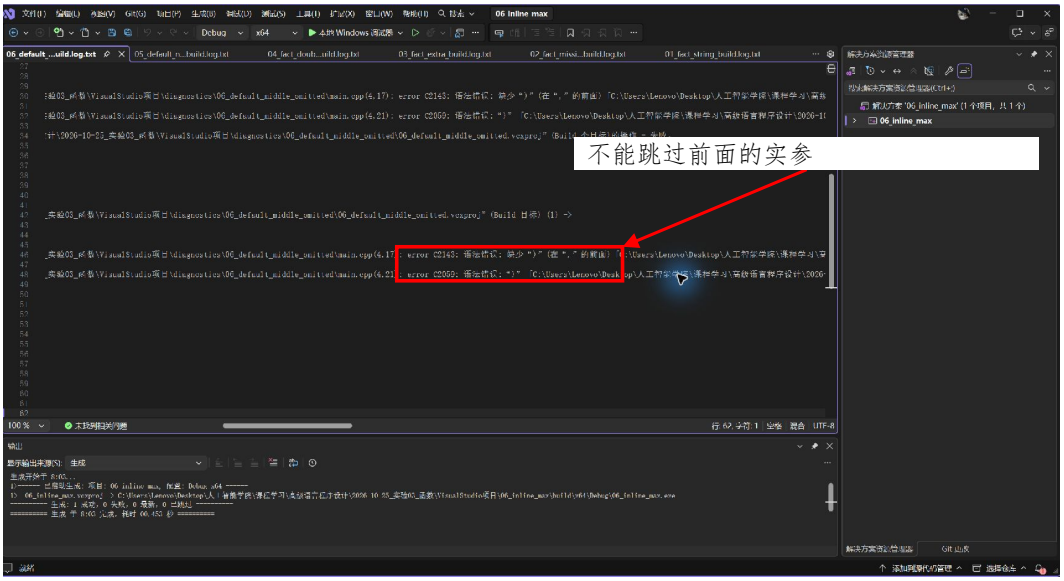


图 37 历史 MSVC 命令行构建日志：Init(,20) 的 C2143/C2059

Init(,20) 在逗号前留空，产生 C2143 和 C2059；调用时只能从最右端连续省略实参。

sum(100)中 num=100、result=5050；sum()中 num=10、result=55。显式实参覆盖默认值，省略实参时使用默认值 10。

默认值须在调用前可见；同一作用域中可以在后续声明或定义处补充此前未给的默认值，但同一形参不能重复指定。若前面的声明已给出默认值，定义处不再重复。

题目 6 课堂练习：内联函数求最大值

使用 inline 函数求两个整数的较大值，输入输出由 main 完成。

lesson::max 用条件运算符返回较大值，两个数相等时返回的数值不变。

解答：代码

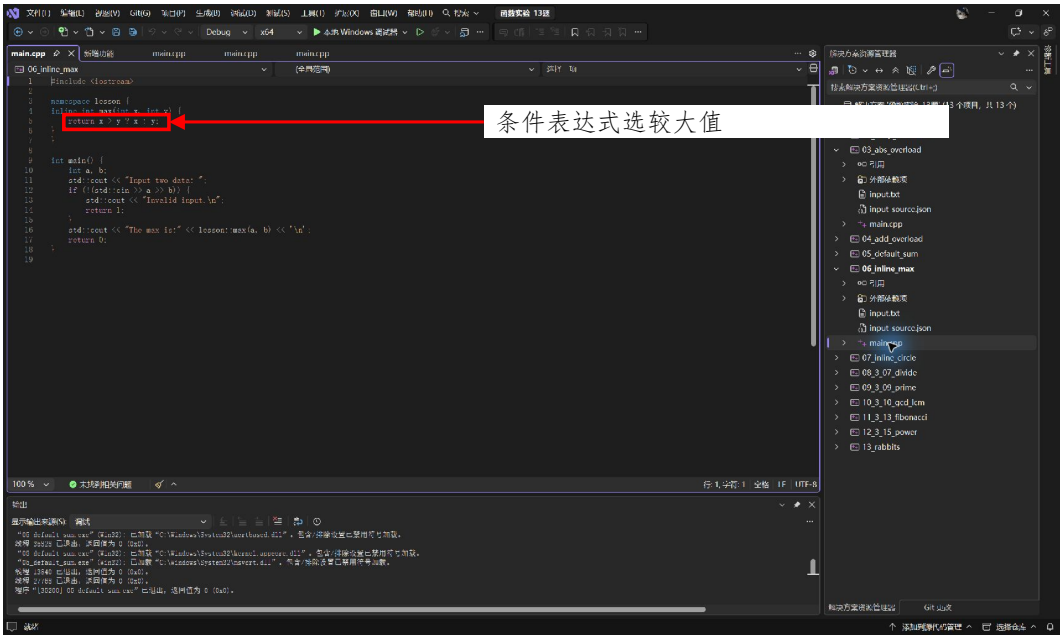


图 38　inline 最大值函数与输入输出

max 用条件表达式返回较大整数，main 负责输入和输出；函数在 main 之前用 inline 定义。

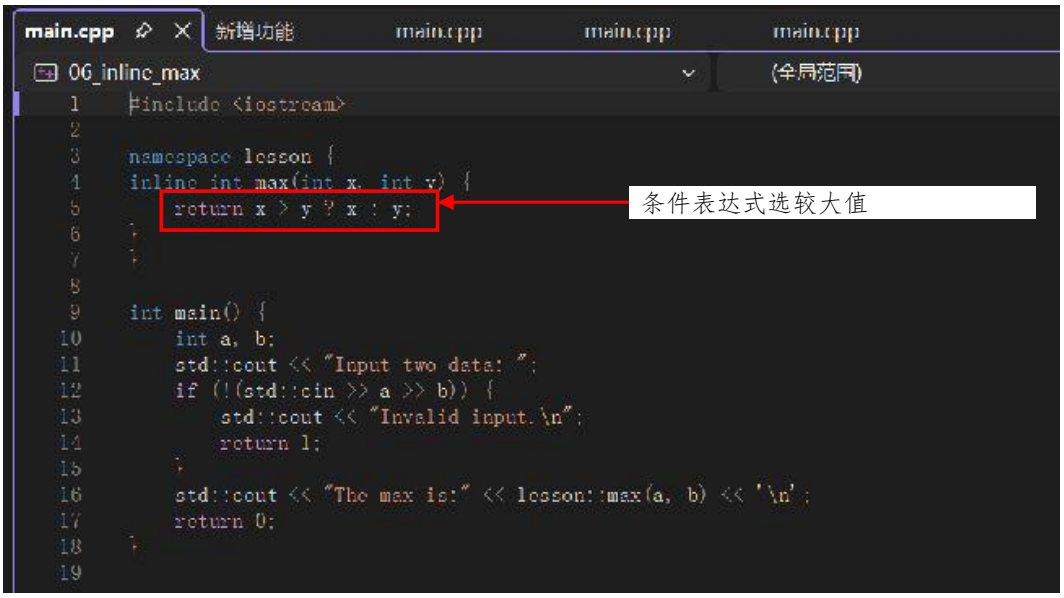


图 39　inline 最大值函数与输入输出

max 用条件表达式返回较大整数，main 负责输入和输出；函数在 main 之前用 inline 定义。

运行结果与调试

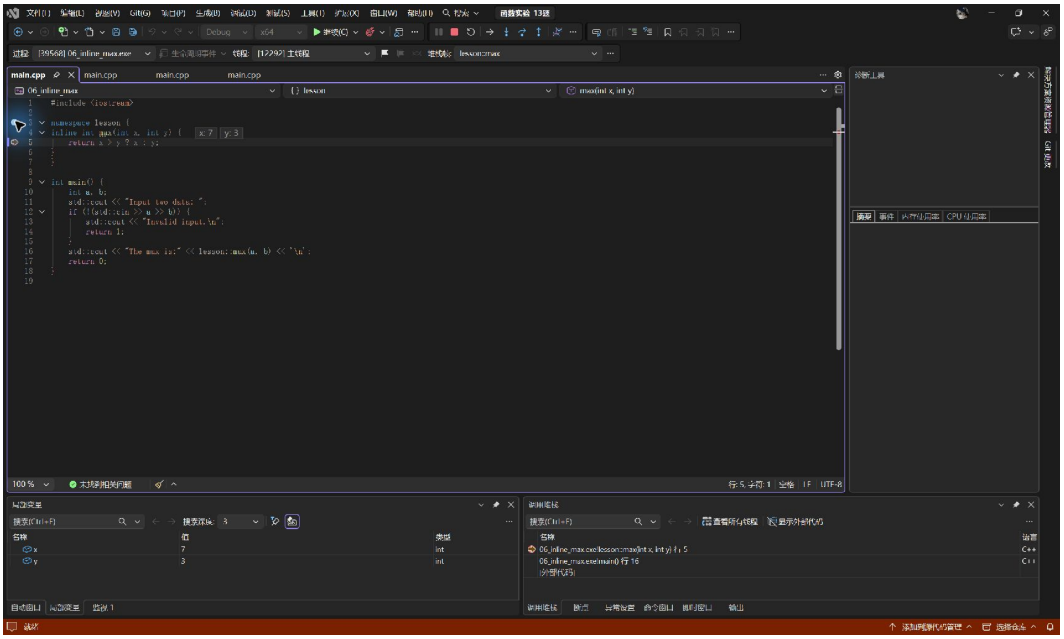


图 40 max 函数中的局部形参

进入 max 时，局部形参 x=7、y=3。

调用堆栈显示当前位于 max，第 5 行由 main 的第 16 行调用。

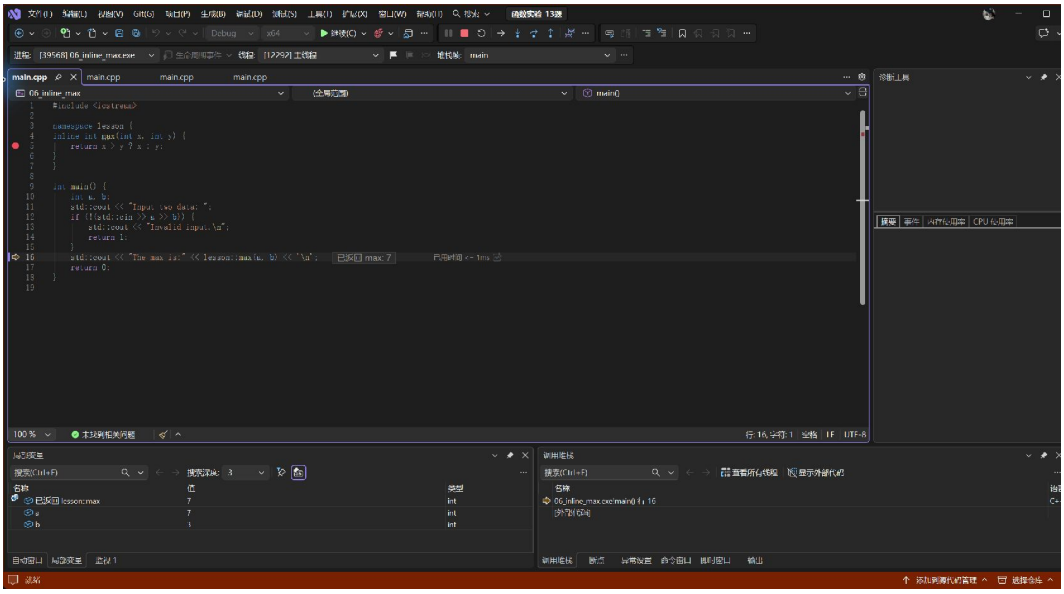


图 41 max 返回后的自动窗口

自动窗口显示 max 的返回值为 7，main 中仍有 a=7、b=3。

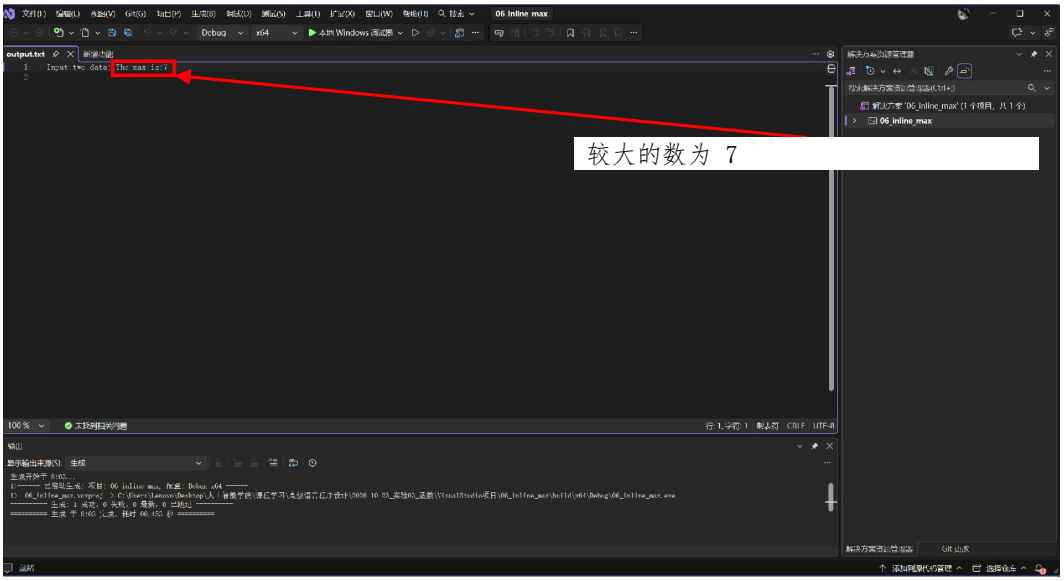


图 42 VS 运行生成的 max 标准输出文件

输入 7 和 3 后，实际生成的 output.txt 显示 The max is:7。

输入 7 和 3 后，max 中的形参 x=7、y=3。

跳出函数后，返回值为 7，执行位置回到 main；output.txt 也显示最大值 7。

inline 关键字不保证函数一定展开，是否展开由编译器决定；本次 Debug 调试仍可进入函数体。

题目 7 课堂练习：内联函数计算圆面积

使用 inline 函数计算圆面积，输出半径为 1、2、3 时的结果。

Circle 使用课件中的近似值 3.1416 计算面积，main 通过循环依次传入半径。

解答：代码

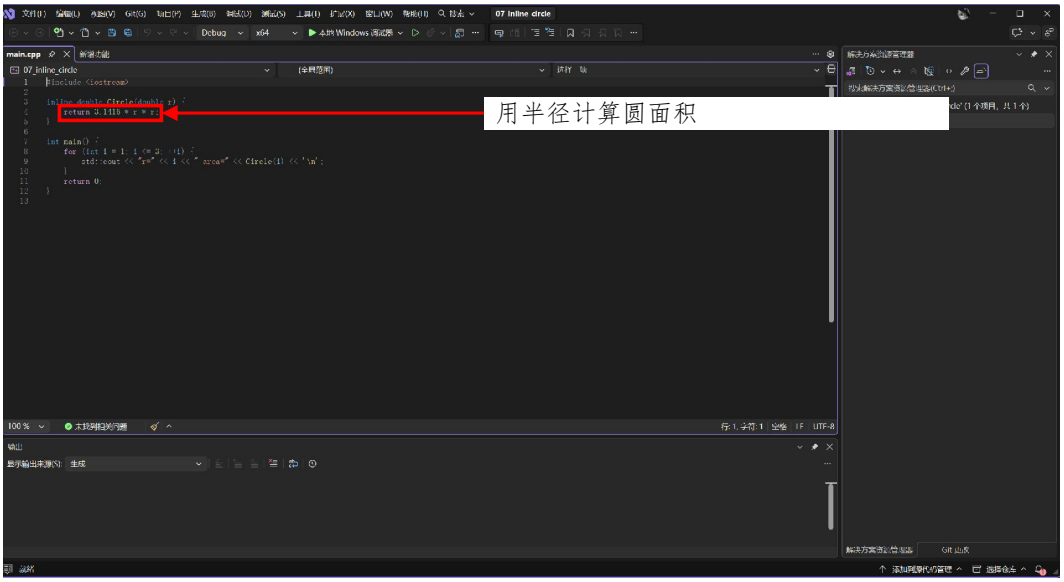


图 43 Circle 函数与三个半径的循环调用

Circle 按 3.1416*r*r 计算圆面积。main 依次传入半径 1、2、3，并输出每次计算结果。

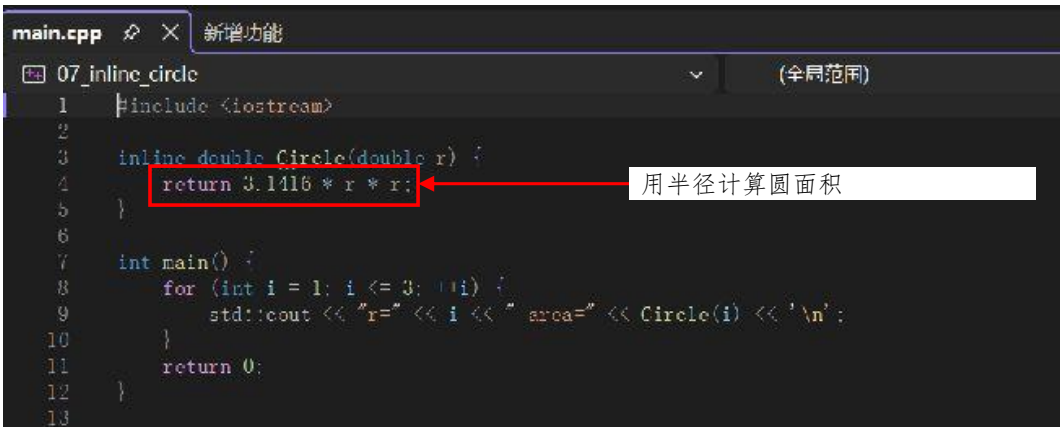


图 44 Circle 函数与三个半径的循环调用

Circle 按 3.1416*r*r 计算圆面积。main 依次传入半径 1、2、3，并输出每次计算结果。

运行结果与调试

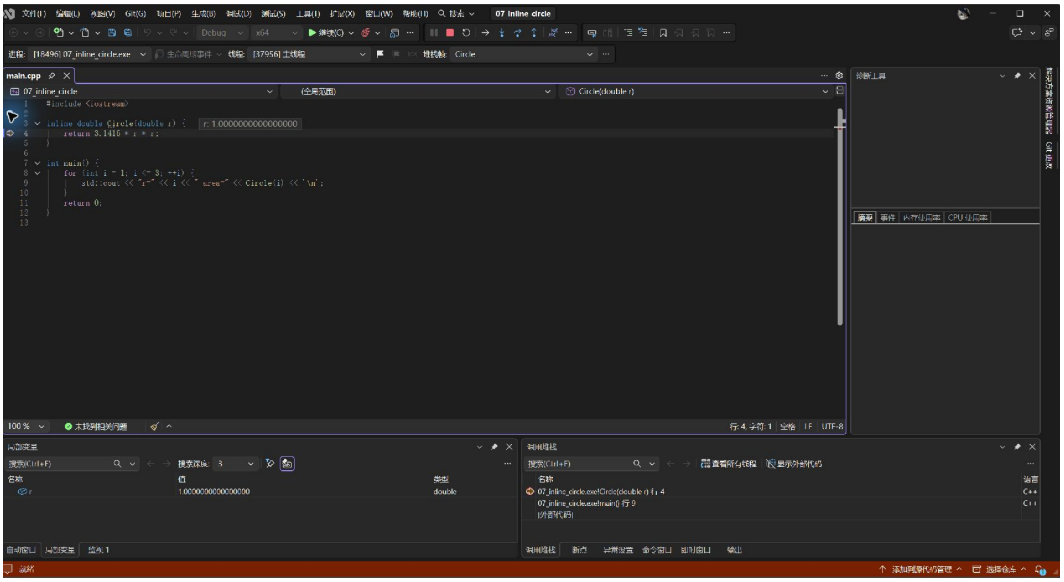


图 45　半径 r=1 的值与类型列

第一次进入 Circle 时，形参 r=1，类型为 double。

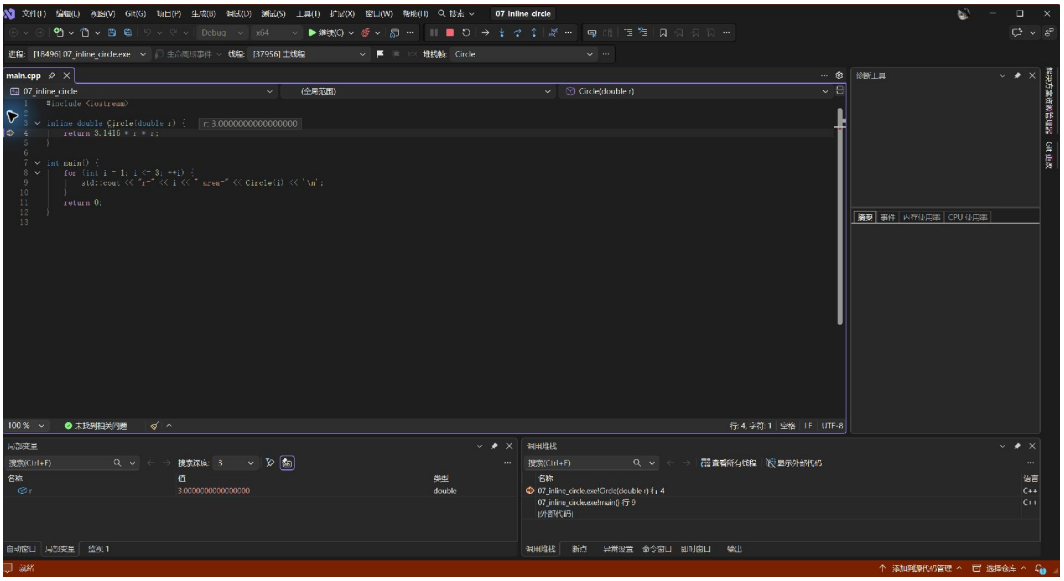


图 46　半径 r=3 的值与类型列

第三次进入同一函数时，形参 r=3，类型仍为 double。

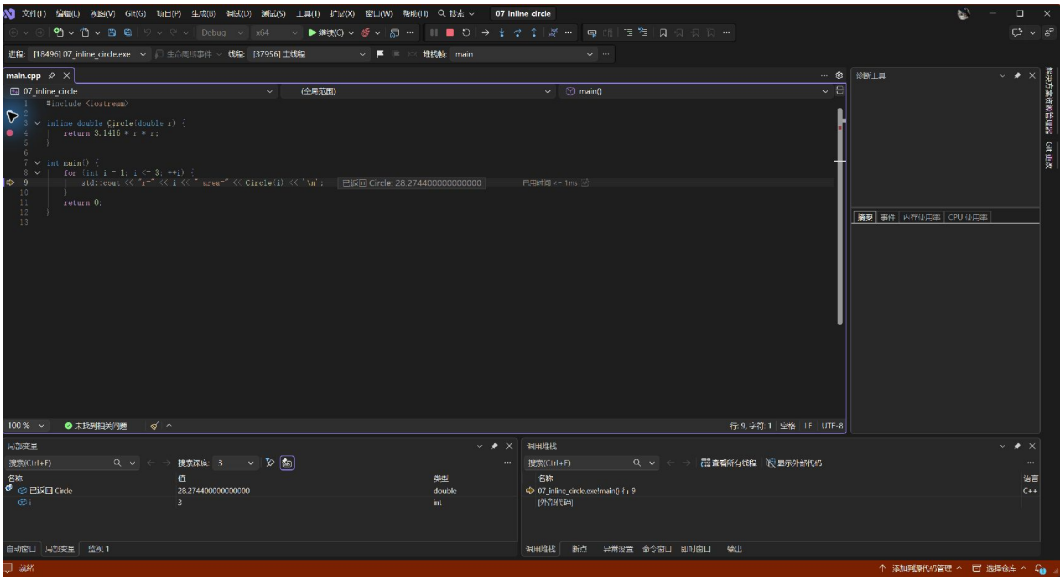


图 47 Circle 返回后的自动窗口

当 i=3 时，自动窗口显示 Circle 的返回值为 28.274400000000000。

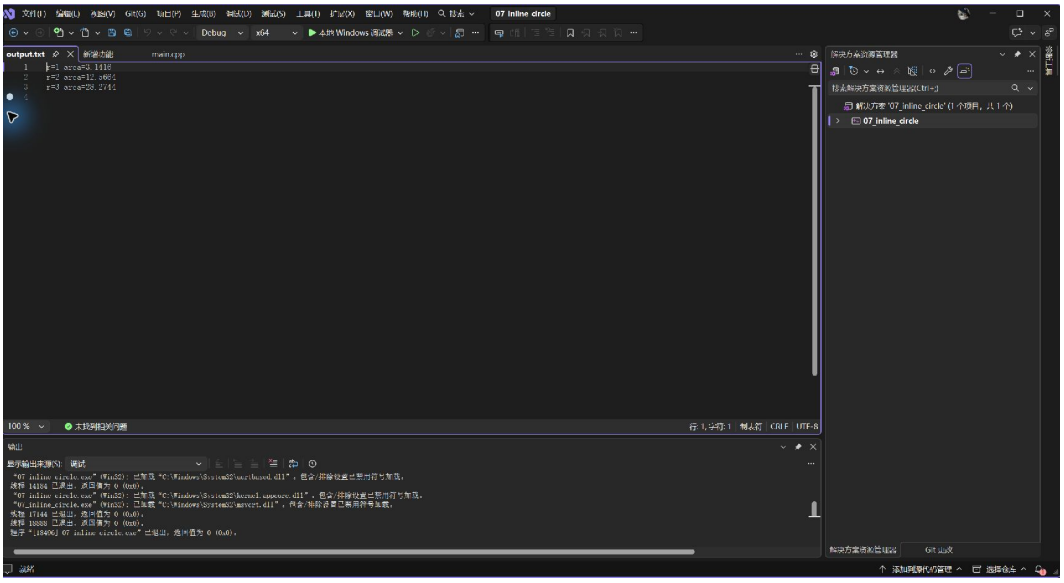


图 48 VS 运行生成的 Circle 标准输出文件

output.txt 中依次显示半径 1、2、3 的面积 3.1416、12.5664、28.2744。

在同一断点依次观察到 r=1、2、3，均为 double。

i=3 时，Circle 返回 28.2744；三次面积输出依次为 3.1416、12.5664、28.2744。

题目 8 教材 3-7：两个无符号短整数相除

函数接收两个 unsigned short int 参数，返回 short int 型整数商；除数为零时返回 -1，在 main 中输入和输出。

main 先检查输入范围。除数非零时还检查整数商能否由 short 表示，再调用题目规定签名的 divide。

解答：代码

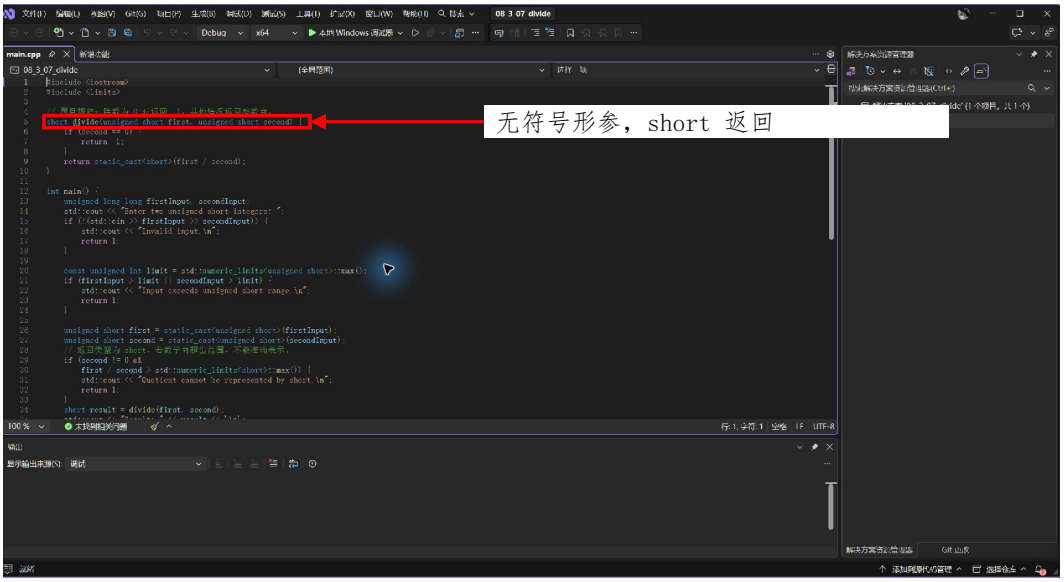


图 49 divide 的函数接口与输入检查

divide 接收两个 unsigned short 参数，返回 short 整数商，除数为零时返回 -1。main 先读取较宽类型的输入，再检查是否超出 unsigned short 范围。

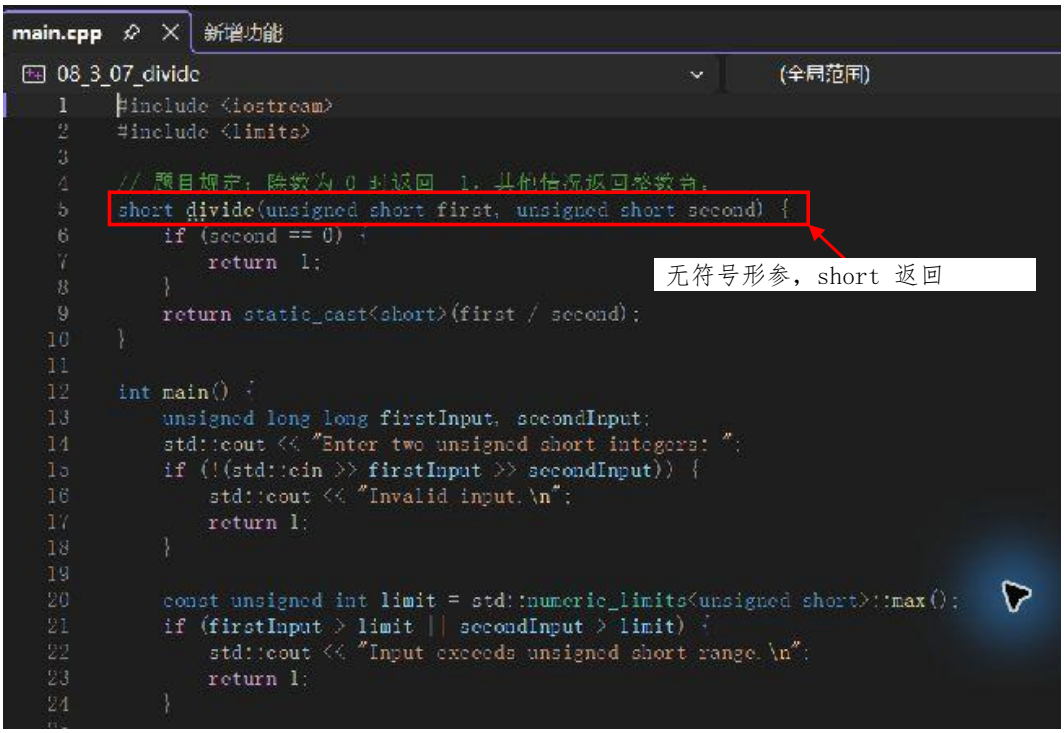


图 50 divide 的函数接口与输入检查

divide 接收两个 unsigned short 参数，返回 short 整数商，除数为零时返回 -1。main 先读取较宽类型的输入，再检查是否超出 unsigned short 范围。

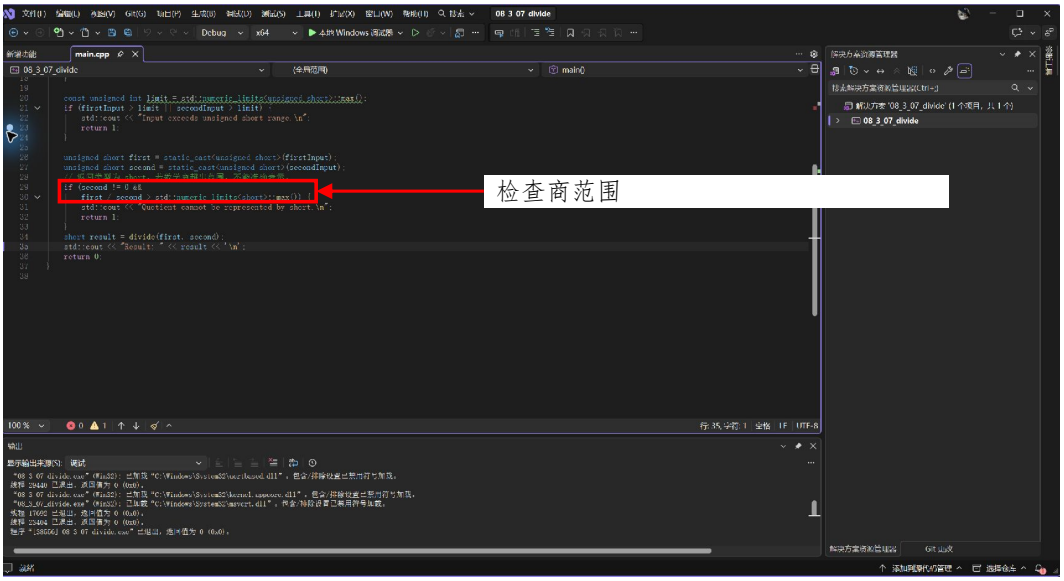


图 51 main 中的范围检查、函数调用与输出

输入通过范围检查后转换为 unsigned short，再检查数学商能否由 short 表示。随后调用 divide，把返回值赋给 result 并输出。

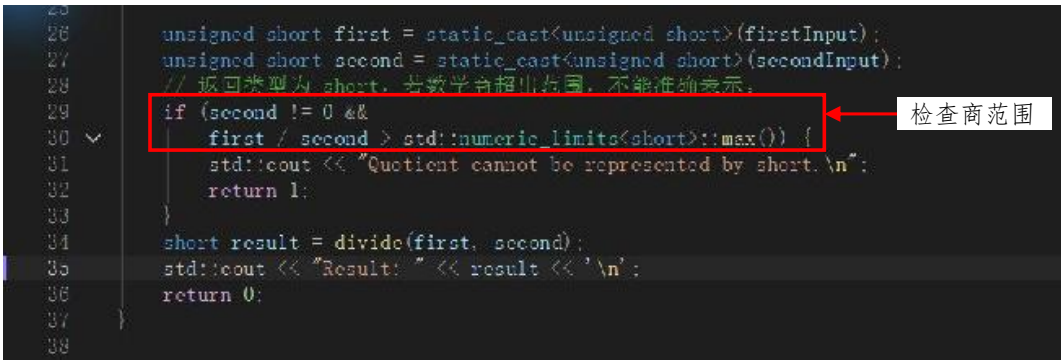


图 52 main 中的范围检查、函数调用与输出

输入通过范围检查后转换为 unsigned short，再检查数学商能否由 short 表示。随后调用 divide，把返回值赋给 result 并输出。

运行结果与调试

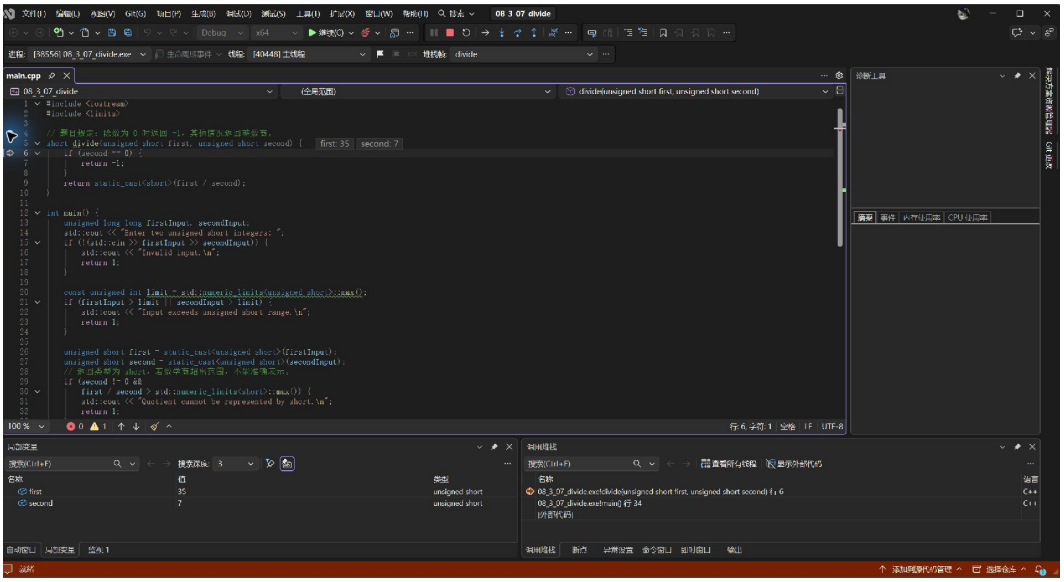


图 53 divide 两个形参的值与类型列

两个形参分别为 first=35、second=7，类型均为 unsigned short。

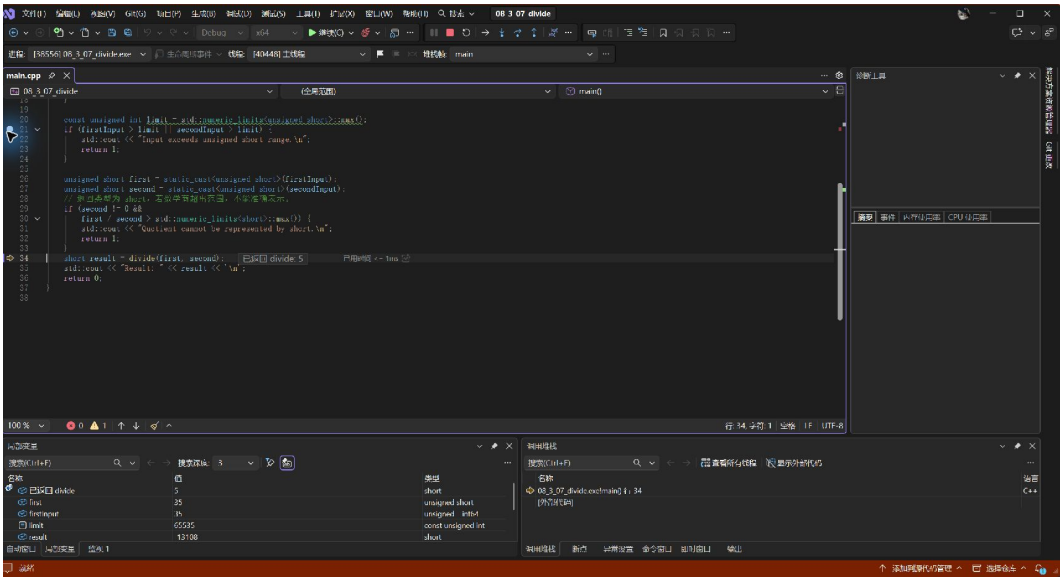


图 54 divide 返回值 5 及 short 类型

跳出 divide 回到 main 第 34 行，VS 显示返回值为 5、类型为 short。

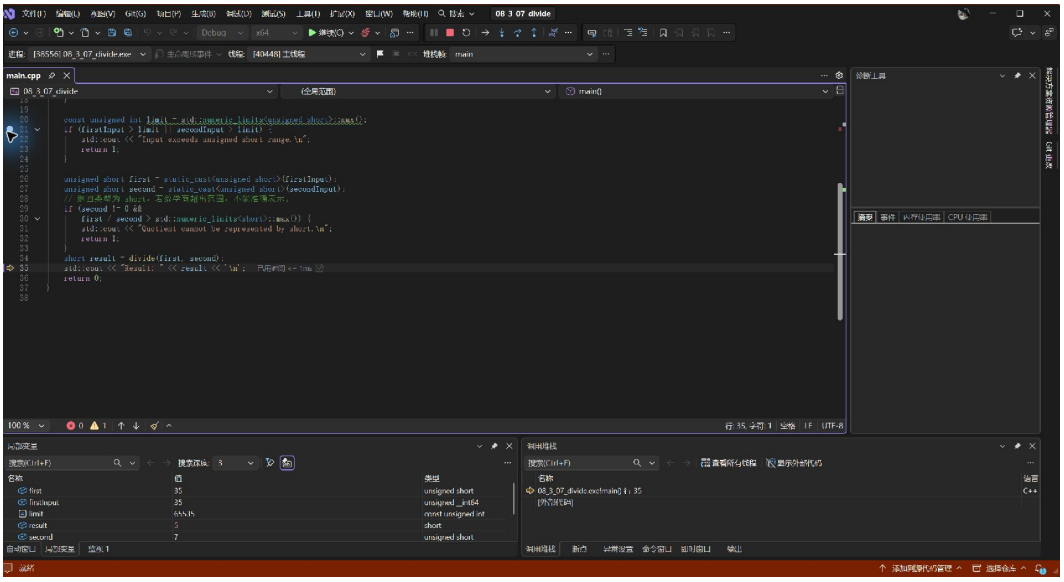


图 55 F10 赋值后 main 中的 result

继续 F10 到第 35 行后，result=5；first=35、second=7 仍可在窗口中核对。

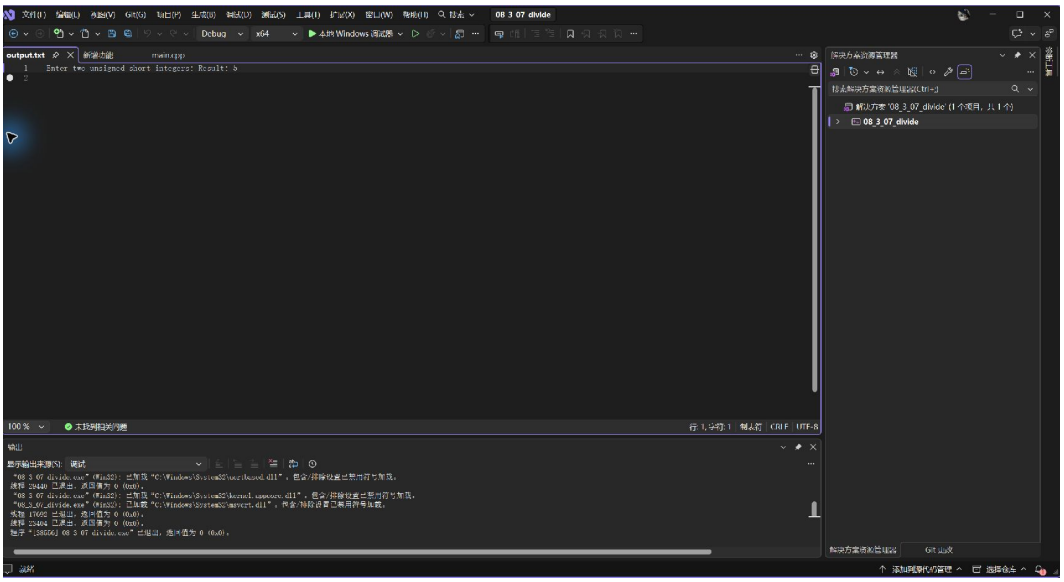


图 56 VS 运行生成的 divide 标准输出文件

以 35 和 7 为样例运行后，生成的 output.txt 中显示 Result: 5。

进入 divide 后，first=35、second=7，形参类型均为 unsigned short。

跳出函数到 main 第 34 行时，VS 显示返回值为 5、类型为 short；执行 F10 到第 35 行，result 接收到 5。

标准输出文件中显示 Result: 5。

题目 9 教材 3-9：判断质数

编写函数判断一个整数是否为质数，在 main 中输入并输出判断结果。

小于 2 的整数返回 false；其余数逐个检查可能的因子，发现整除因子就返回 false。循环条件写为 divisor <= number/divisor，避免乘法溢出。

解答：代码

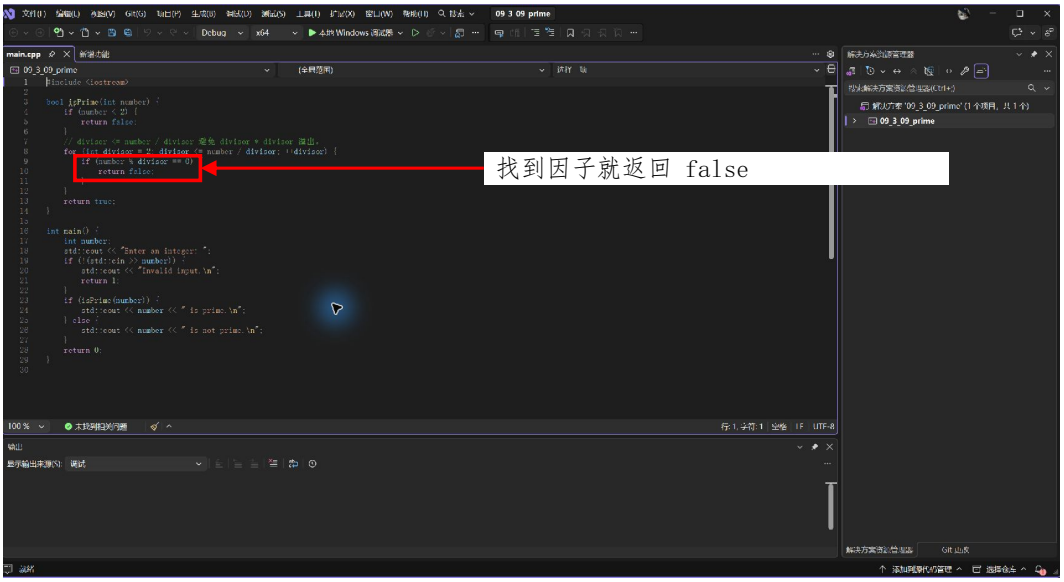


图 57　质数判断函数与 main 的输入输出

isPrime 对小于 2 的数直接返回 false，再依次尝试除数，找到因子就返回 false。main 根据布尔返回值输出判断结果。

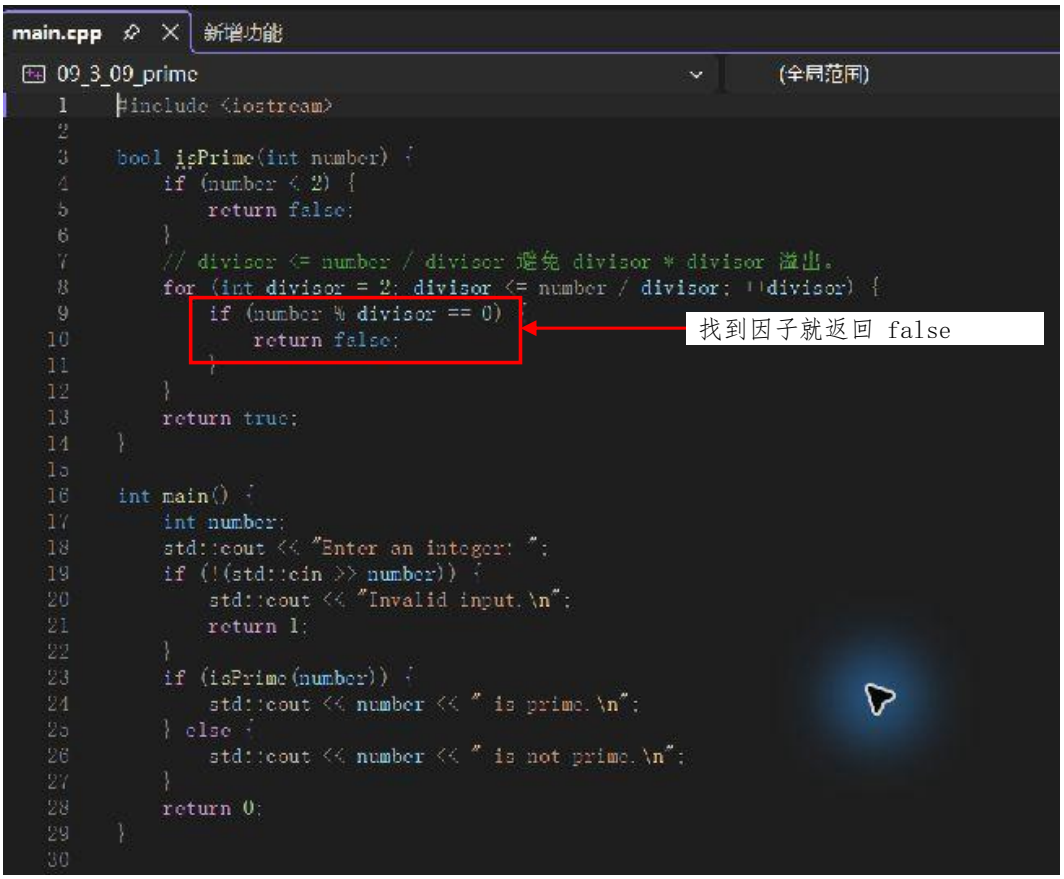


图 58　质数判断函数与 main 的输入输出

isPrime 对小于 2 的数直接返回 false，再依次尝试除数，找到因子就返回 false。main 根据布尔返回值输出判断结果。

运行结果与调试

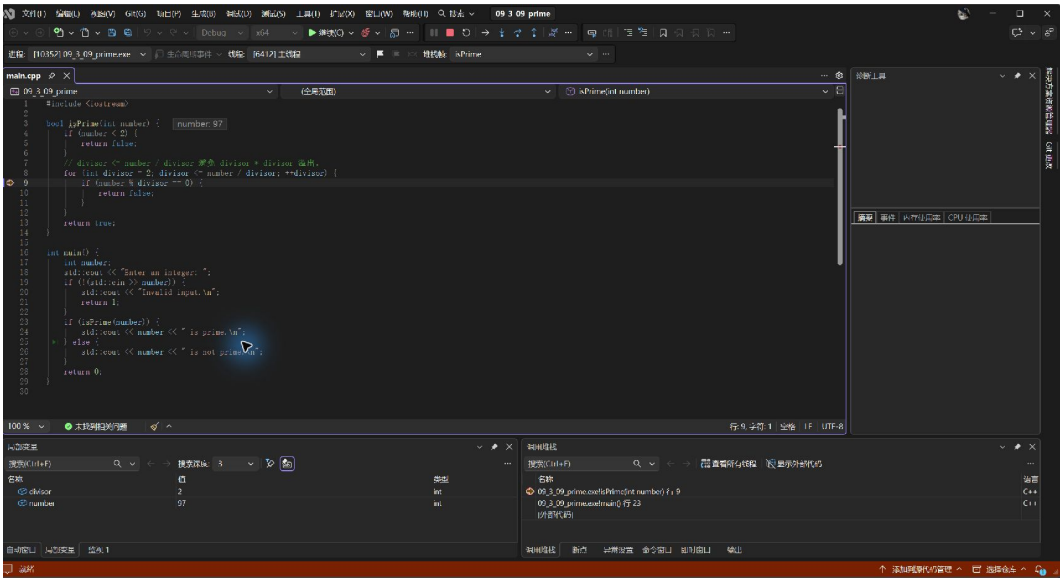


图 59　第一次断点中的 number 与 divisor

第一次停在第 9 行时，number=97、divisor=2。

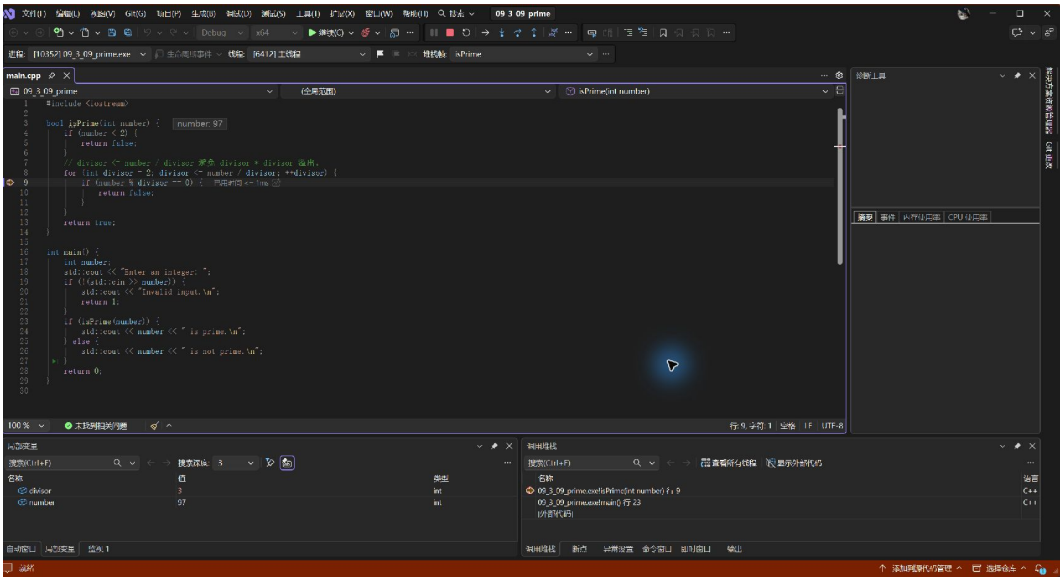


图 60　再次命中断点时的 divisor=3

尝试跳出函数时再次被同一断点打断，此时 number=97、divisor=3。

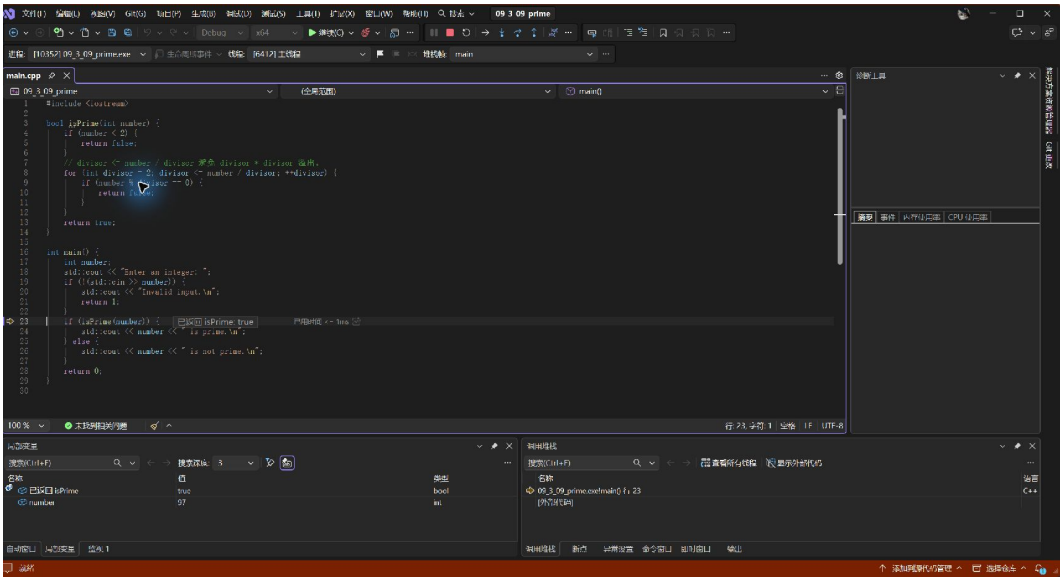


图 61　isPrime 返回后的自动窗口

移除断点后跳出函数，自动窗口显示 isPrime 返回 true，number 仍为 97。

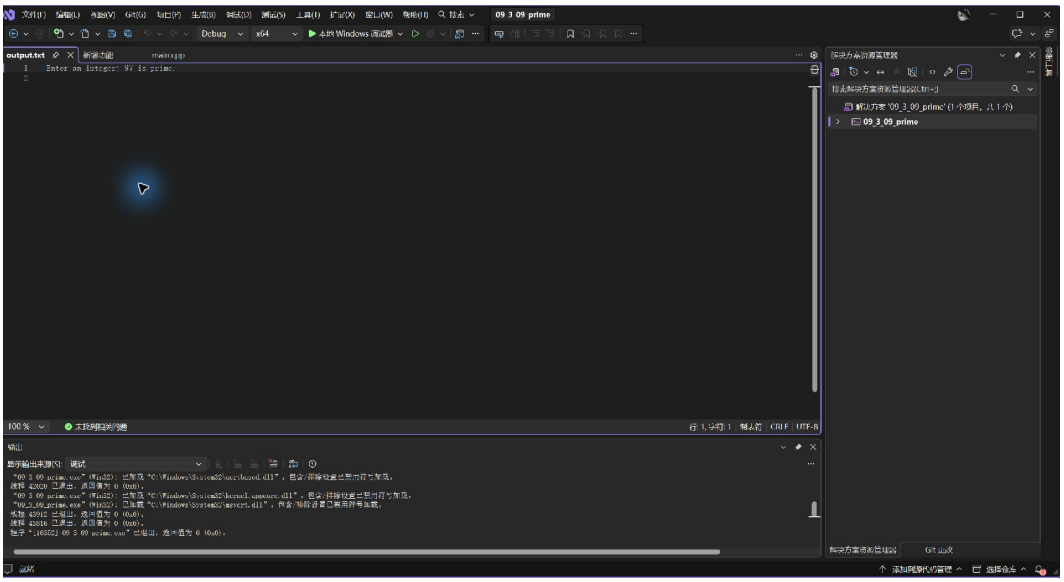


图 62　VS 运行生成的 prime 标准输出文件

output.txt 中显示“97 is prime.”，与本次返回值 true 一致。

在第 9 行的两次断点中，number 保持 97，divisor 依次为 2、3。移除断点后跳出 isPrime，返回 true。

标准输出判断 97 为质数。

题目 10　教材 3-10：最大公约数和最小公倍数

分别编写函数求两个整数的最大公约数与最小公倍数，输入和结果输出放在 main 中。

最大公约数使用辗转相除法；最小公倍数先除以最大公约数再乘另一数。输入为 int，计算使用 long long，并处理负数与零。

解答：代码

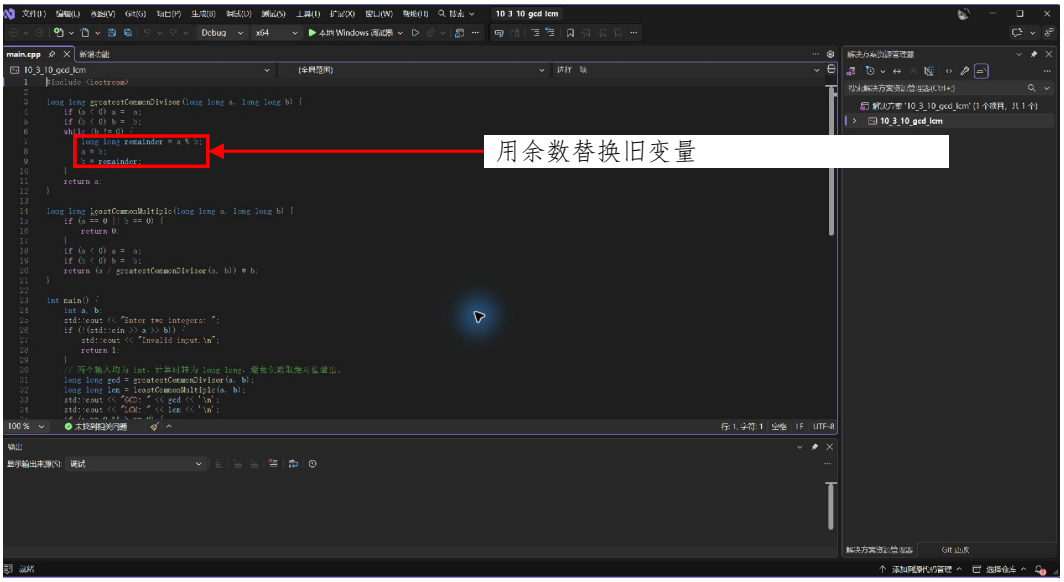


图 63　最大公约数与最小公倍数函数

最大公约数用辗转相除法，把除数和余数替换为下一轮的两个数。最小公倍数先除以最大公约数再相乘，零值和负数在对应分支中处理。

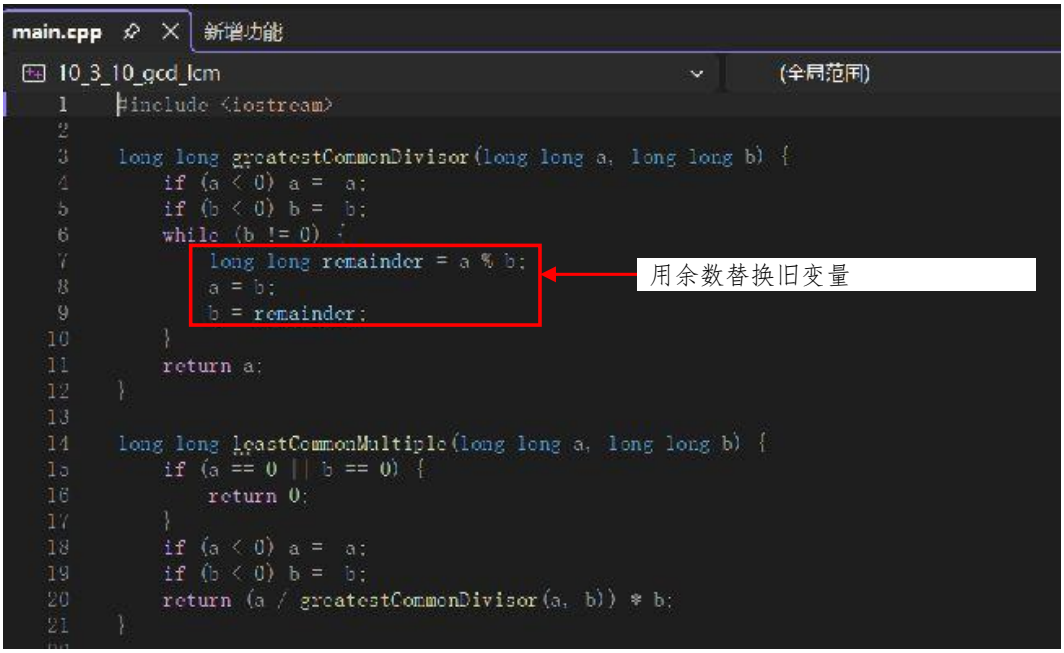


图 64　最大公约数与最小公倍数函数

最大公约数用辗转相除法，把除数和余数替换为下一轮的两个数。最小公倍数先除以最大公约数再相乘，零值和负数在对应分支中处理。

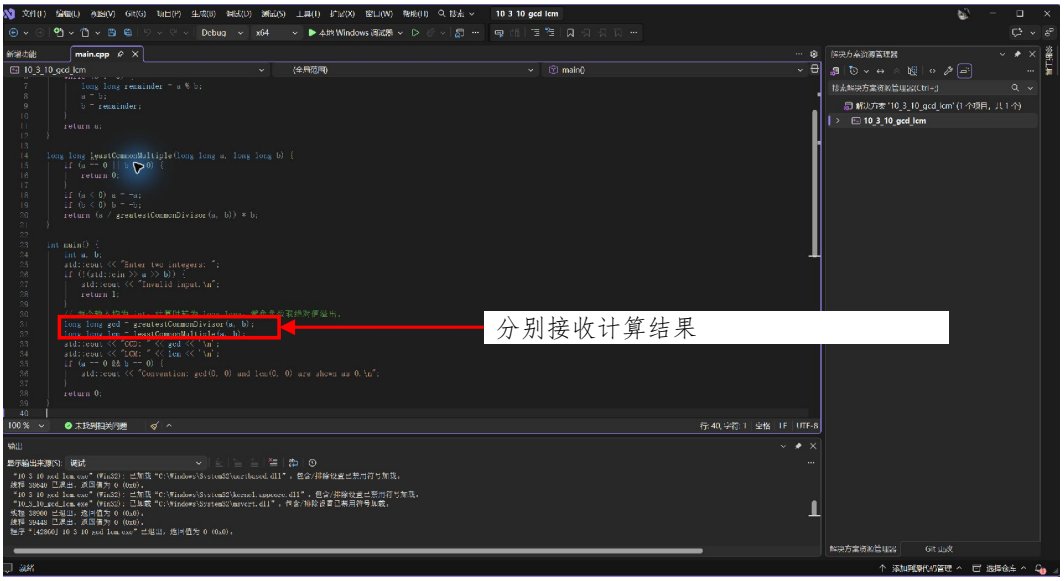


图 65 main 中的两次调用与结果输出

main 读入两个整数，分别调用两个函数，并将结果接收到 gcd、lcm 中。两个输入均为零时，额外输出本程序的约定说明。

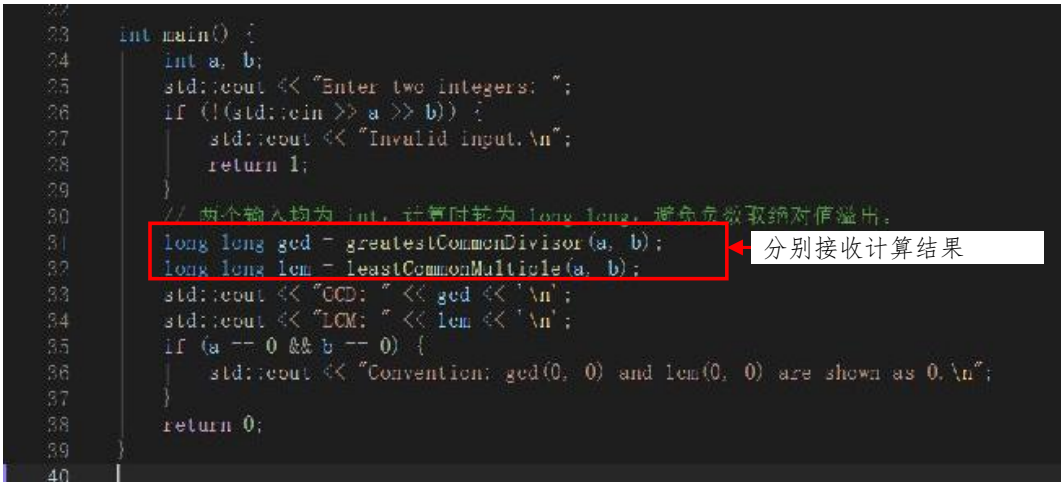


图 66 main 中的两次调用与结果输出

main 读入两个整数，分别调用两个函数，并将结果接收到 gcd、lcm 中。两个输入均为零时，额外输出本程序的约定说明。

运行结果与调试

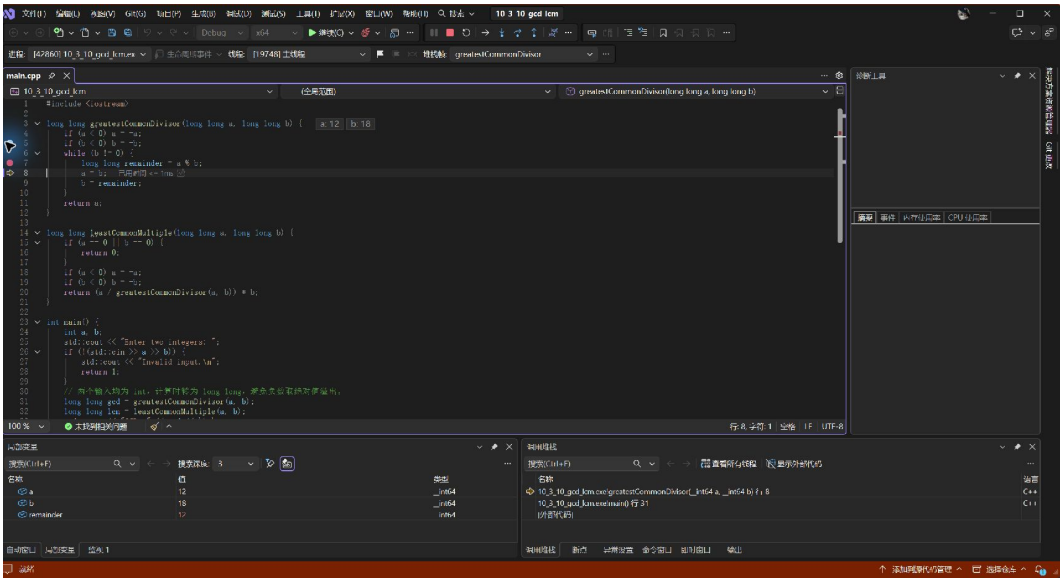


图 67　计算余数后的局部变量

执行 remainder=a%b 后，a=12、b=18，remainder=12。

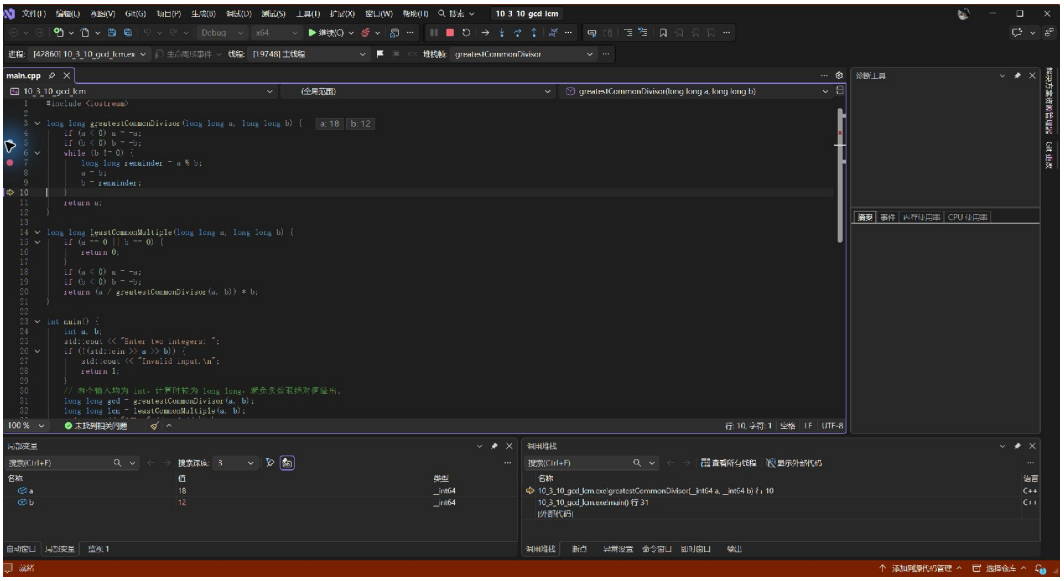


图 68　完成两次替换后的 a 与 b

执行 a=b 和 b=remainder 后，a=18、b=12，作为下一轮计算的输入。

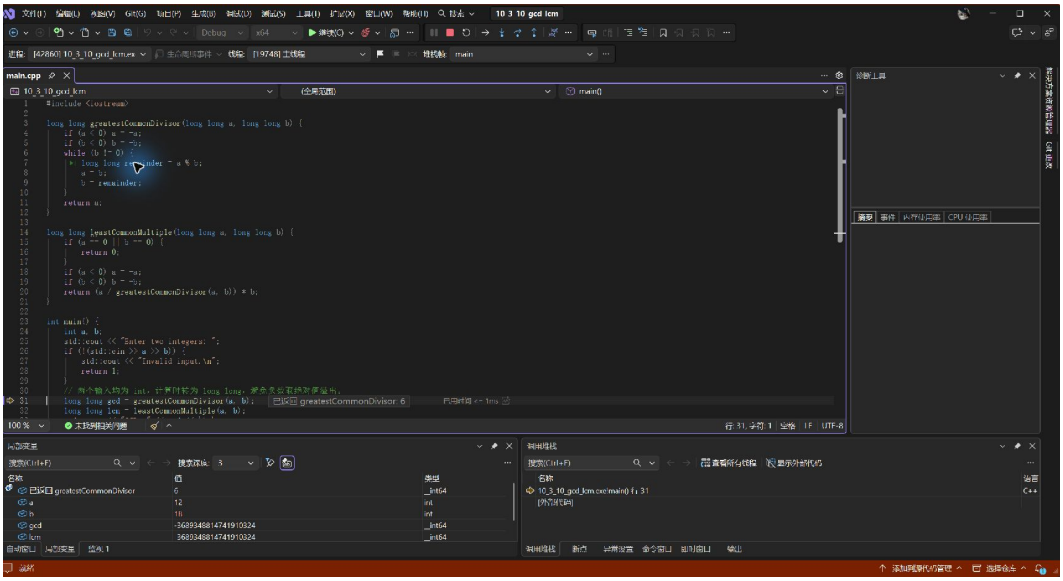


图 69 greatestCommonDivisor 的返回值 6

跳出函数回到 main 第 31 行，VS 显示返回值 6。

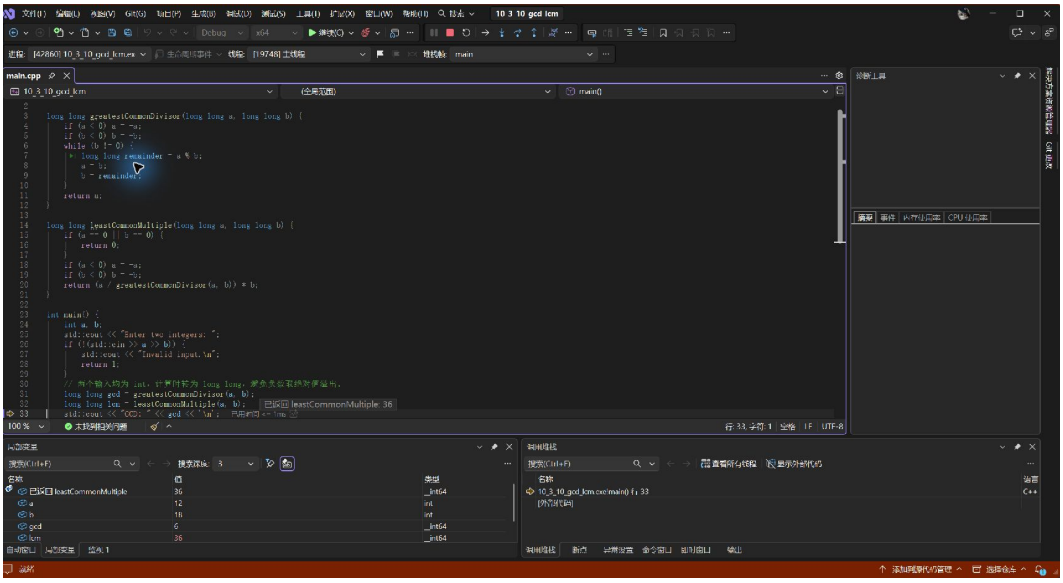


图 70 main 接收的公约数与公倍数

接收两个返回值后,gcd=6、lcm=36,自动窗口还显示 leastCommonMultiple 返回 36。

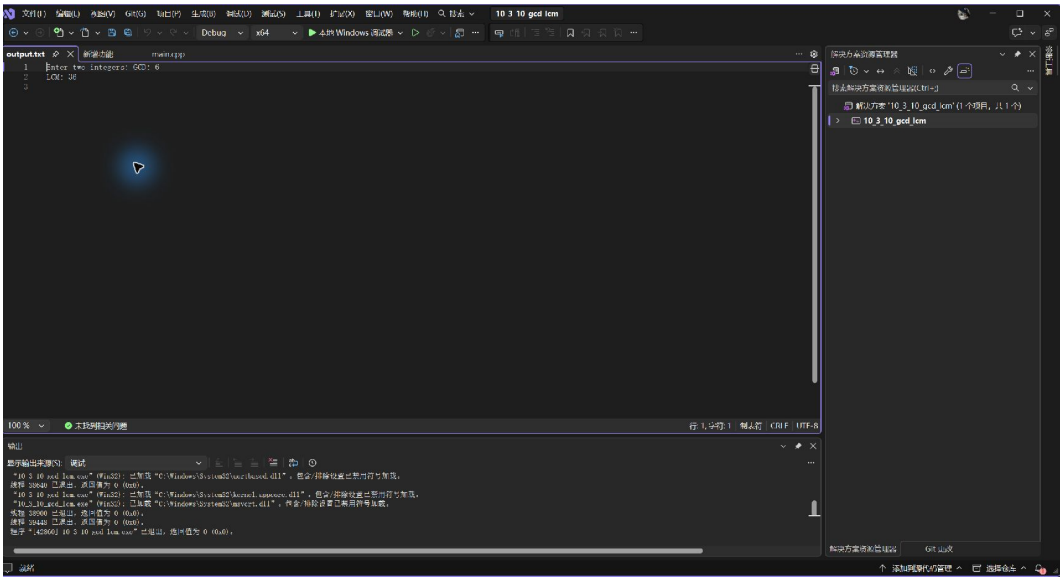


图 71 VS 运行生成的 gcd_lcm 标准输出文件

以 12 和 18 为样例运行后，output.txt 中输出 GCD: 6 和 LCM: 36。

计算余数后，a=12、b=18、remainder=12；执行两次替换后，a=18、b=12。

greatestCommonDivisor 返回 6，随后 main 接收到 gcd=6；完成第二个函数调用后，lcm=36。

标准输出分别为 GCD: 6、LCM: 36。

题目 11 教材 3-13：递归斐波那契级数

按 F1=F2=1、Fn=F(n-1)+F(n-2) 编写递归函数，并观察递归调用过程。

n 为 1 或 2 时返回 1，其余项递归求前两项之和。main 输出前 n 项和第 n 项，并把演示范围限制为 1 至 30。

解答：代码

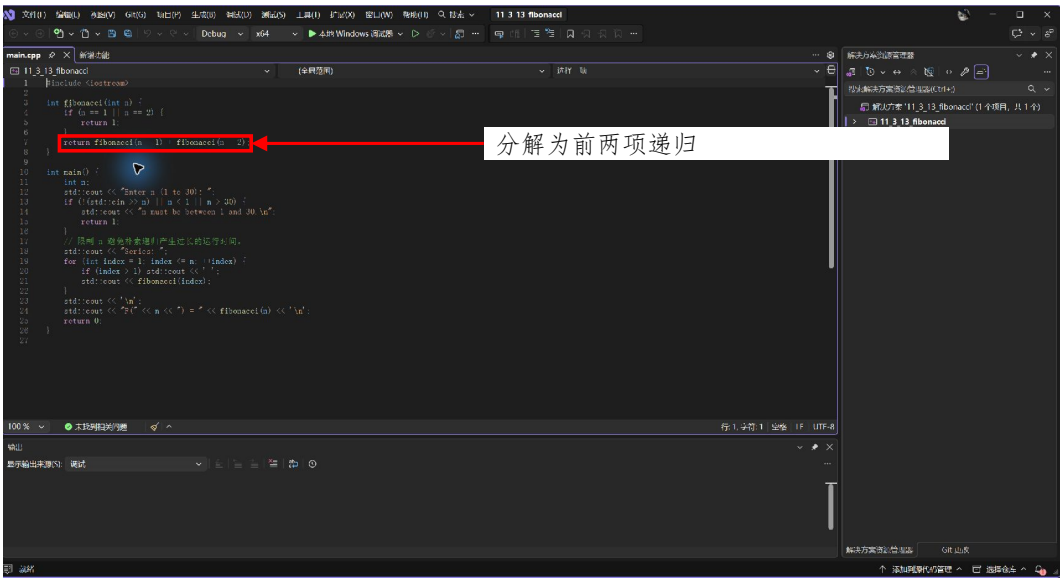


图 72 递归斐波那契函数与序列输出

第 1、2 项直接返回 1，其余项递归求前两项的和。main 限制 n 的范围，依次输出前 n 项，再输出第 n 项。

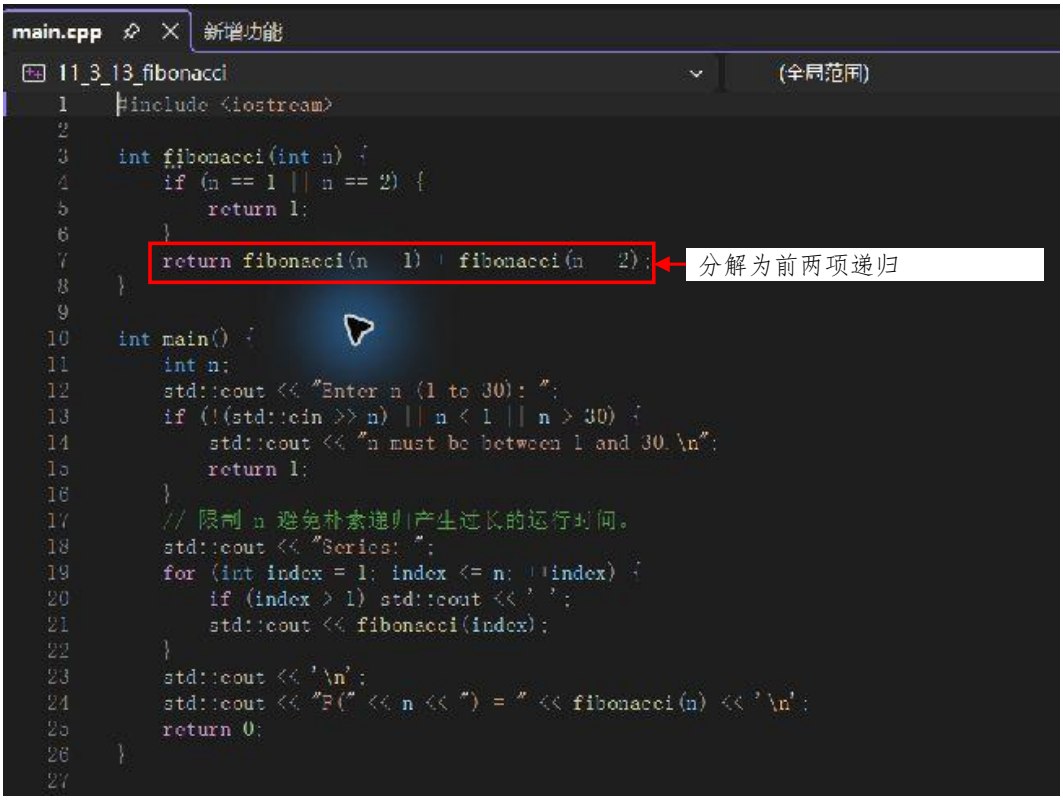


图 73 递归斐波那契函数与序列输出

第 1、2 项直接返回 1，其余项递归求前两项的和。main 限制 n 的范围，依次输出前 n 项，再输出第 n 项。

运行结果与调试

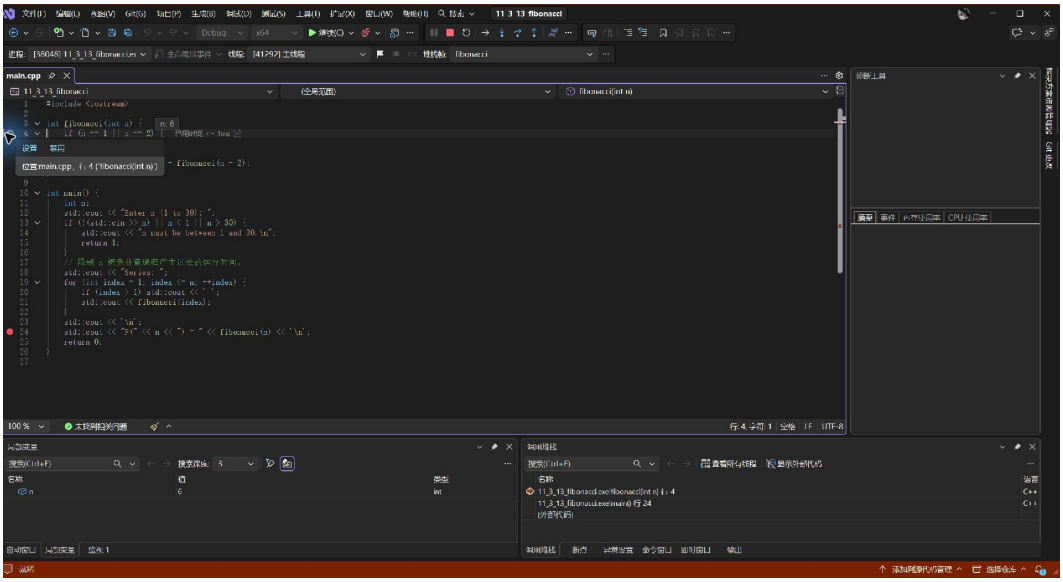


图 74 本次最终递归调用的 n=6

最终 fibonacci(n)调用命中断点时，局部形参 n=6。

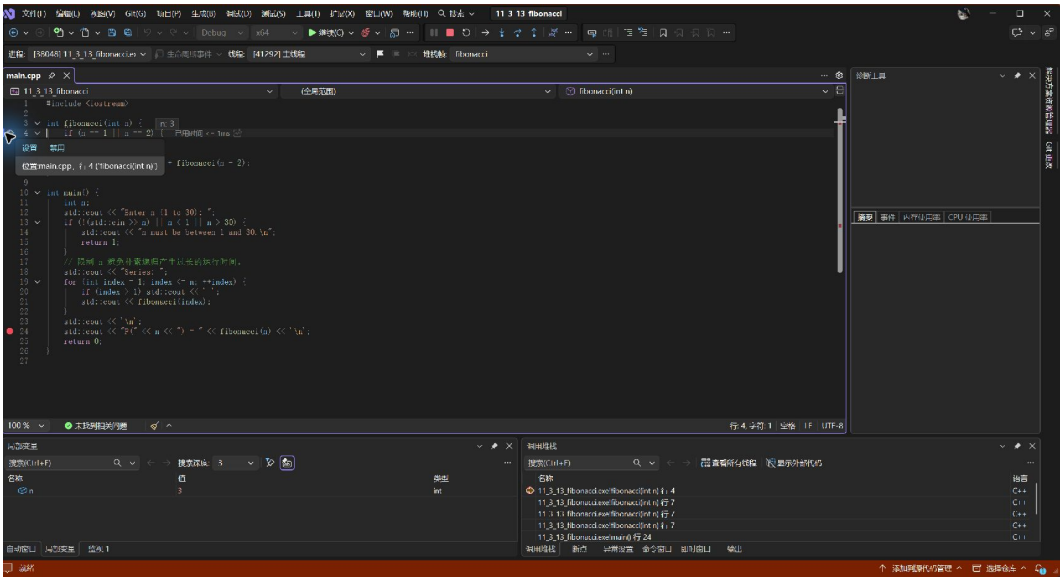


图 75　n=3 暂停点的多层递归调用堆栈

在 n=3 的暂停点，堆栈中可见多层 fibonacci 调用，下方为 main 第 24 行。

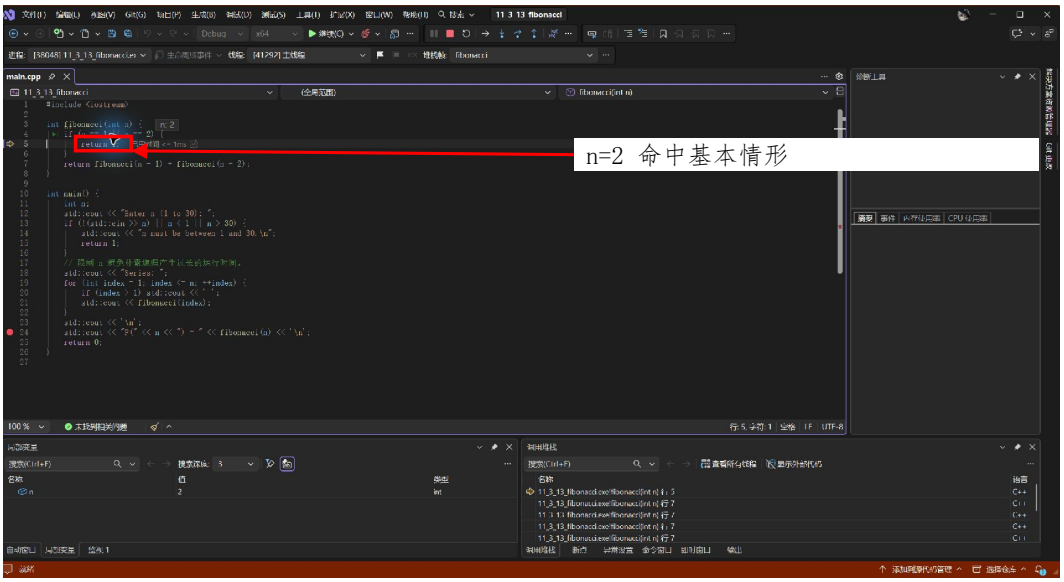


图 76　n=2 时停在基本情形 return 1

n=2 满足基本条件，当前停在第 5 行，准备返回 1。

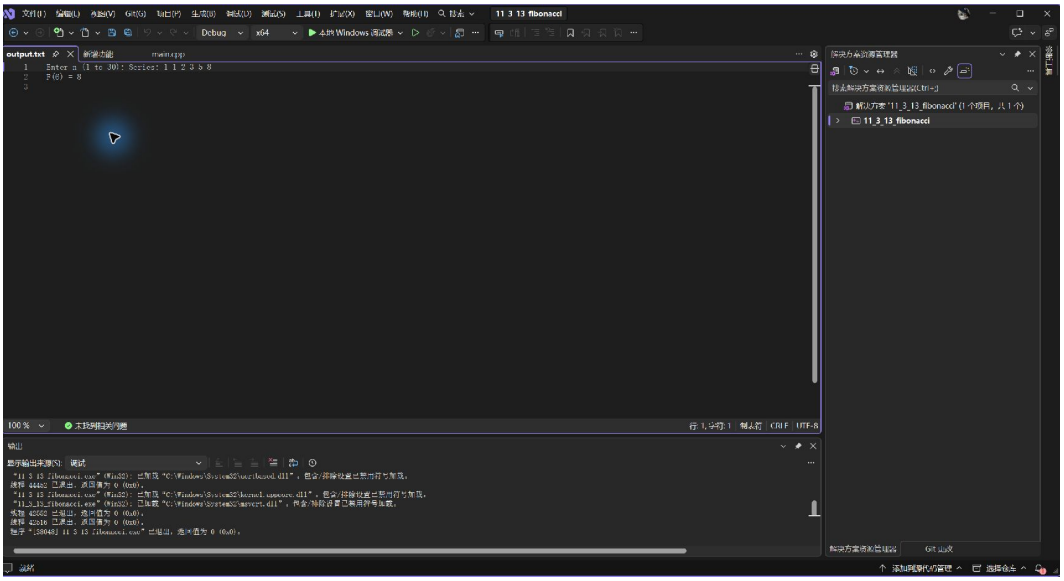


图 77 VS 运行生成的 fibonacci 标准输出文件

output.txt 中的序列为 1 1 2 3 5 8，并显示 F(6)=8。

本次输入 n=6。继续命中递归断点后，n=3 时已形成多层调用；n=2 进入基本情形，准备返回 1。

程序输出前 6 项 1 1 2 3 5 8，第 6 项为 8。

题目 12 教材 3-15：递归重载求幂

在同一程序中实现两个递归 getPower 重载，分别接收 int 和 double 底数；整型版本在负指数时返回 0。

两个重载都以指数为 0 为终止条件。实数版本的负指数逐层加 1，并在返回时除以底数。要让整数 a 调用实数重载，使用 static_cast<double>(a)。

解答：代码

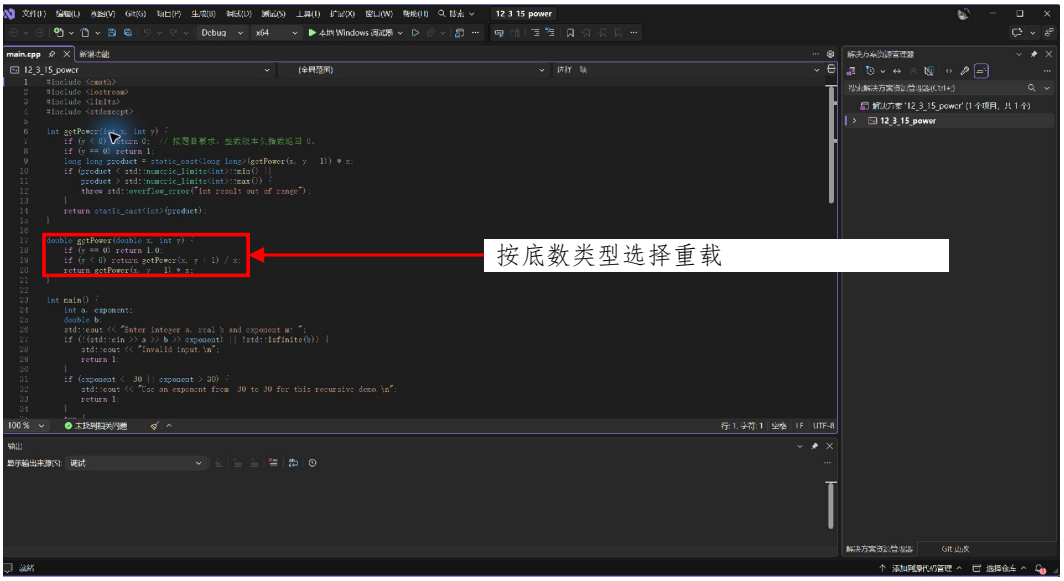


图 78 两个 getPower 重载与输入检查

两个重载按底数类型区分，通过递归使指数趋向 0，整数负指数按题目返回 0，实数版本支持负指数。main 先读入 a、b 和指数，并检查范围。

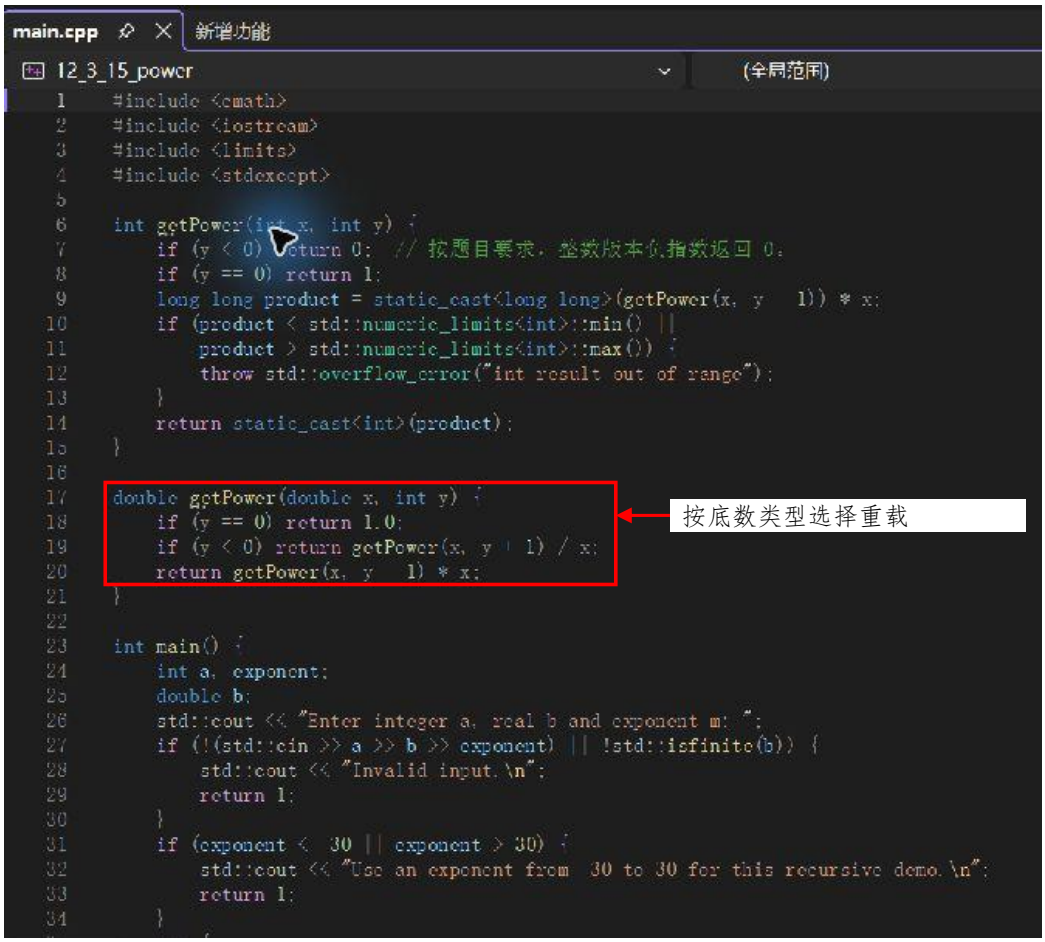


图 79　两个 getPower 重载与输入检查

两个重载按底数类型区分，通过递归使指数趋向 0，整数负指数按题目返回 0，实数版本支持负指数。main 先读入 a、b 和指数，并检查范围。

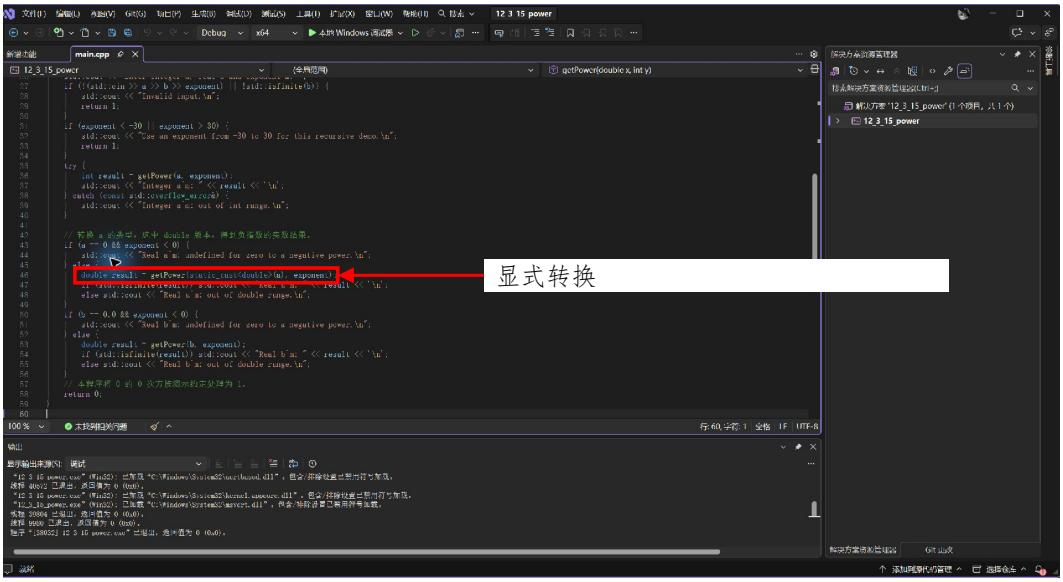


图 80　main 中的求幂调用及异常处理

main 分别调用整数版本和 double 版本，对零底数负指数、整数溢出及非有限结果作处理。static_cast<double>(a) 使整数 a 进入实数重载。

```
    try {
        int result = getPower(a, exponent);
        std::cout << "Integer a^m: " << result << '\n';
    } catch (const std::overflow_error&) {
        std::cout << "Integer a^m: out of int range.\n";
    }

    // 转换 a 的类型，选中 double 版本，得到负指数的实数结果。
    if (a == 0 && exponent < 0) {
        std::cout << "Real a^m: undefined for zero to a negative power.\n";
    } else {
        double result = getPower(static_cast<double>(a), exponent);
        if (std::isfinite(result)) std::cout << "Real a^m: " << result << '\n';
        else std::cout << "Real a^m: out of double range.\n";
    }
    if (b == 0.0 && exponent < 0) {
        std::cout << "Real b^m: undefined for zero to a negative power.\n";
    } else {
        double result = getPower(b, exponent);
        if (std::isfinite(result)) std::cout << "Real b^m: " << result << '\n';
        else std::cout << "Real b^m: out of double range.\n";
    }
    // 本程序将 0 的 0 次方按惯例约定处理为 1。
    return 0;
}
```

显式转换

图 81　main 中的求幂调用及异常处理

main 分别调用整数版本和 double 版本，对零底数负指数、整数溢出及非有限结果作处理。static_cast<double>(a) 使整数 a 进入实数重载。

运行结果与调试

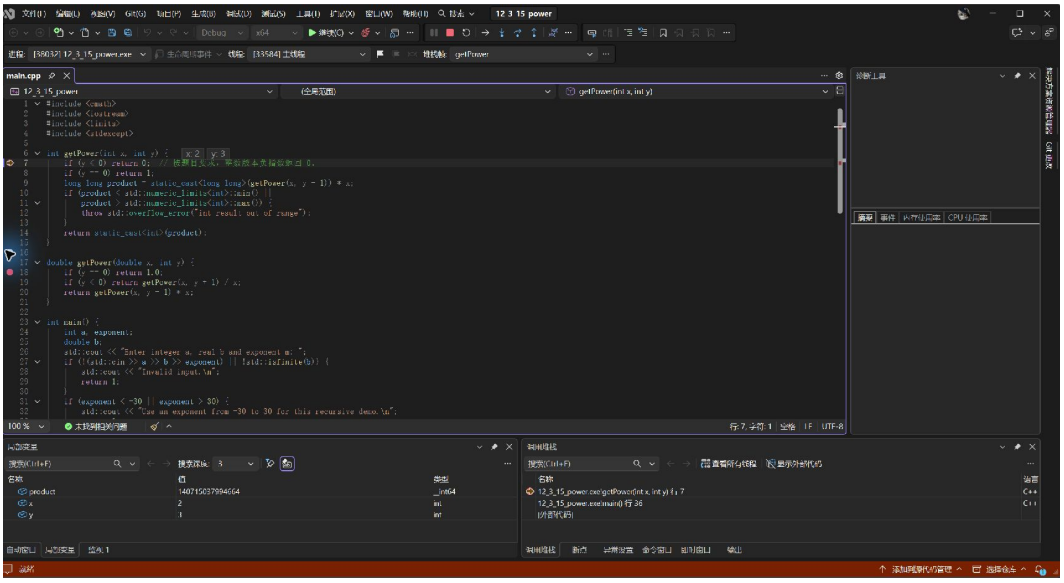


图 82　整数重载中的 x=2、y=3

整数重载中的形参 x=2、y=3。

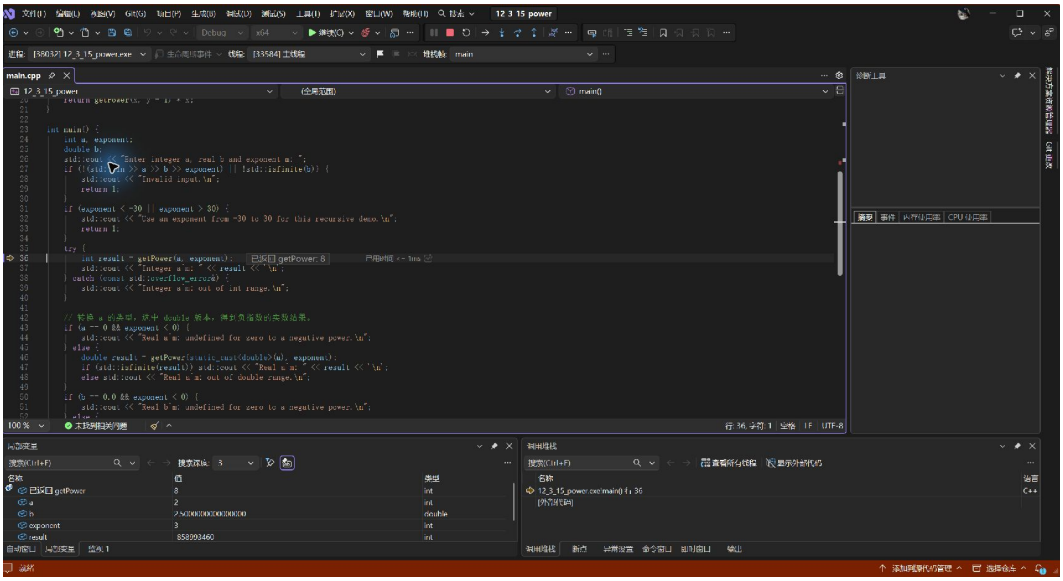


图 83 整数 getPower 的返回值 8

跳出整数重载后，VS 显示返回值 8。

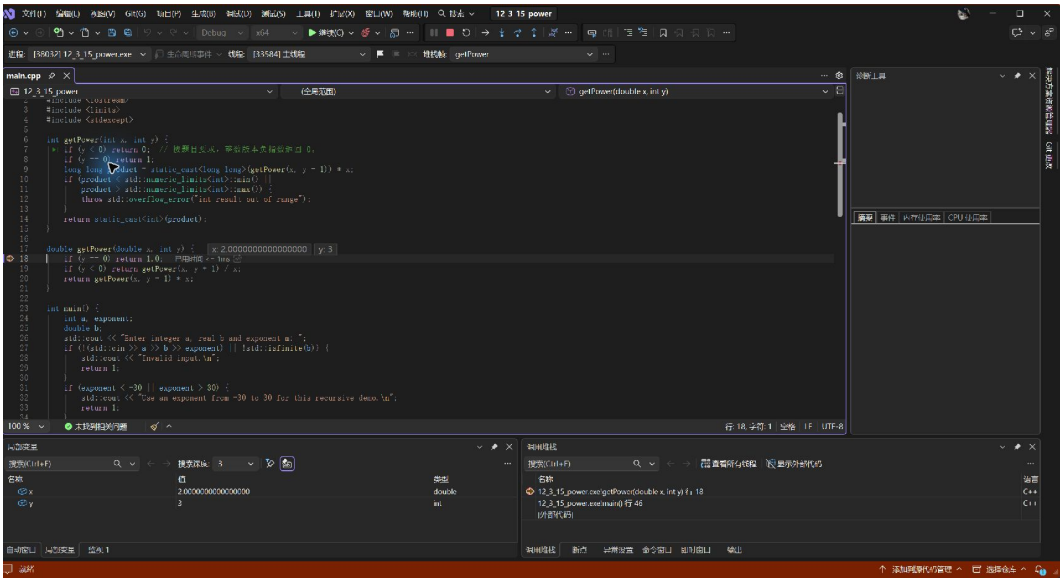


图 84 实数重载形参的值与类型列

底数 x=2.0、类型为 double，指数 y=3、类型为 int，对应 static_cast<double>(a)的调用。

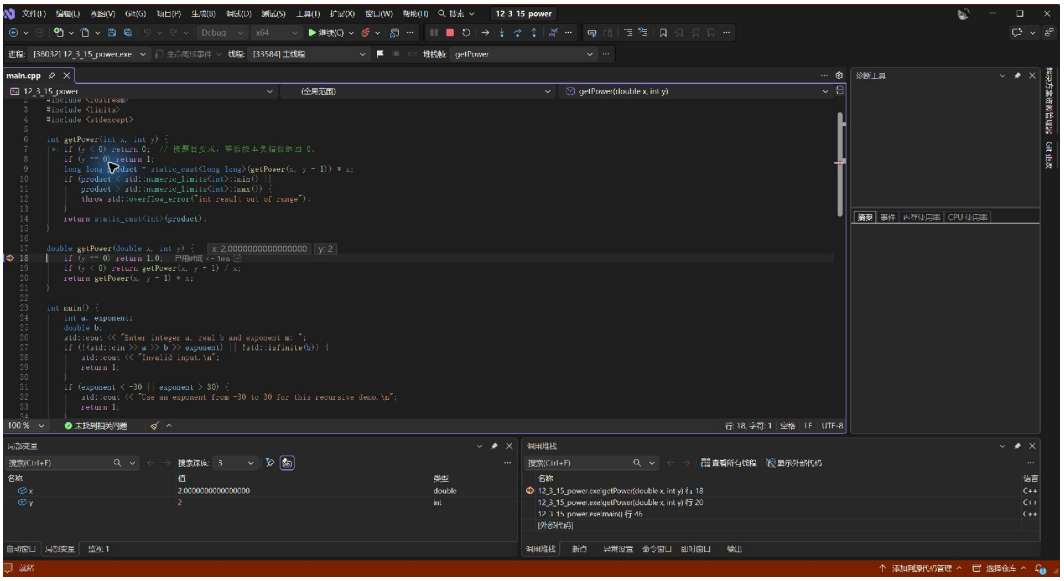


图 85　下一层 double 递归中的指数 2

下一次递归暂停时，底数仍为 2.0，指数 y 已从 3 变为 2。

堆栈显示两层 getPower(double,int)，下方的 main 第 46 行对应 static_cast<double>(a)。

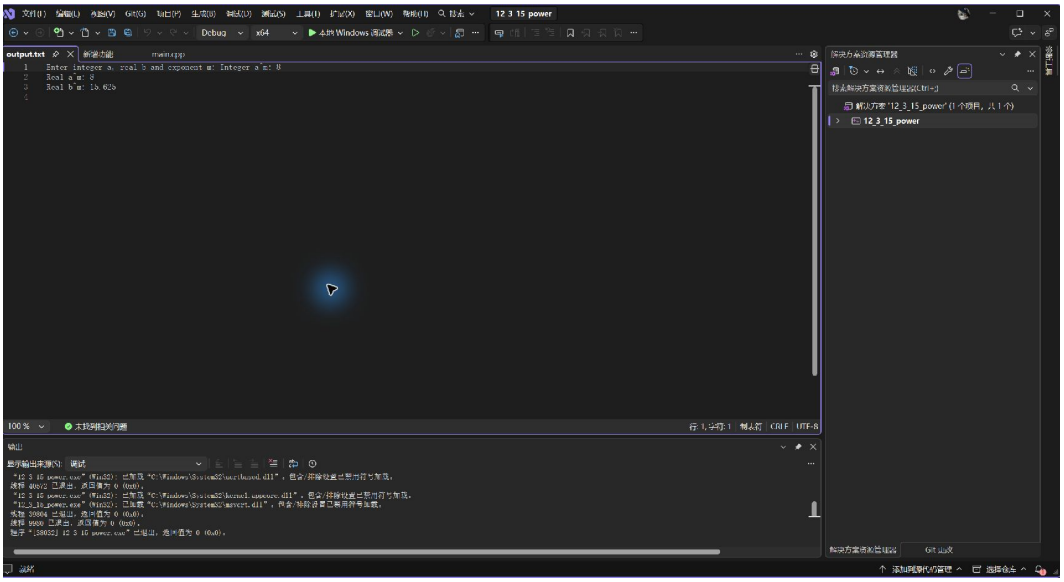


图 86　VS 运行生成的 power 标准输出文件

输入 2、2.5、3 后，整数 a 的结果为 8，实数 a 的结果为 8，实数 b 的结果为 15.625。

输入 a=2、b=2.5、m=3，整数重载的形参为 2、3，返回值为 8。

static_cast<double>(a)进入 double 重载，形参为 2.0、3；下一层指数变为 2，堆栈可见两层调用。

标准输出中 a 的整数和实数结果均为 8，b 的实数结果为 15.625。

题目 13　补充题：前 20 个月的兔子总数

最初有一对兔子，从出生第 3 个月起，每对兔子每月生一对；新出生的兔子也按同样规律繁殖，假设兔子都不死。求前 20 个月每月的兔子总只数，在 main 中用循环实现。

用 previousPairs 和 currentPairs 保存相邻两月的对数。每轮输出后先算 nextPairs，再依次更新前两项；将对数乘 2 得到只数。

解答：代码

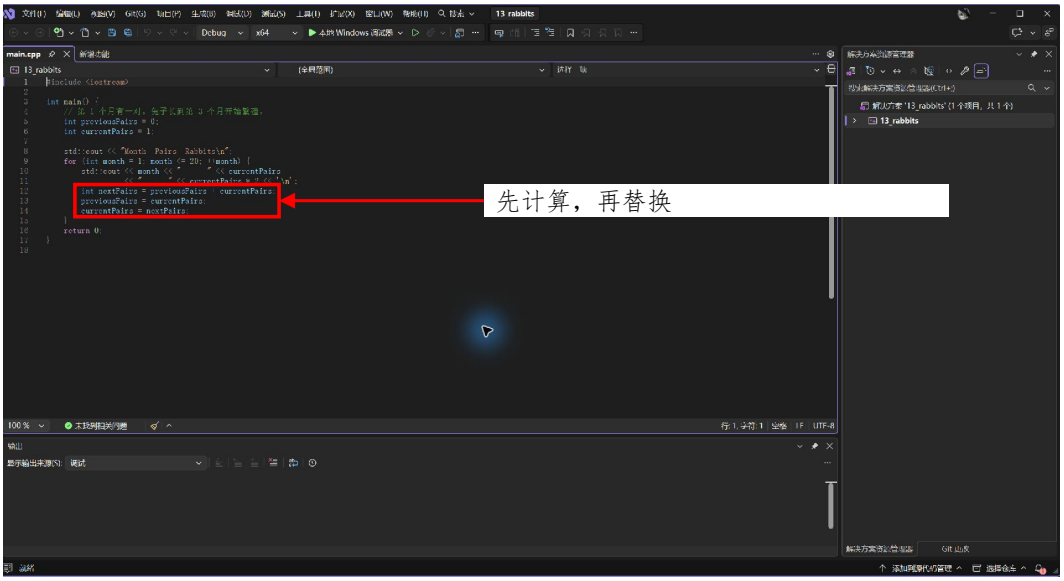


图 87　main 内循环计算每月兔子数量

previousPairs 和 currentPairs 保存相邻两月的对数，输出时乘 2 得到只数。每轮先输出当前月，再计算 nextPairs，依次替换两个旧变量；不另写求解函数。

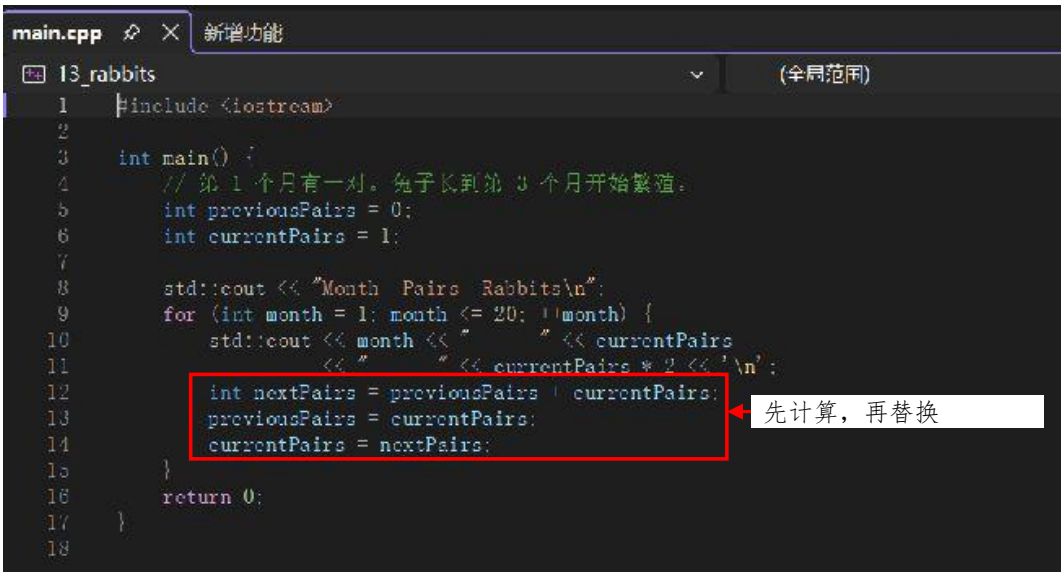


图 88 main 内循环计算每月兔子数量

previousPairs 和 currentPairs 保存相邻两月的对数，输出时乘 2 得到只数。每轮先输出当前月，再计算 nextPairs，依次替换两个旧变量；不另写求解函数。

运行结果与调试

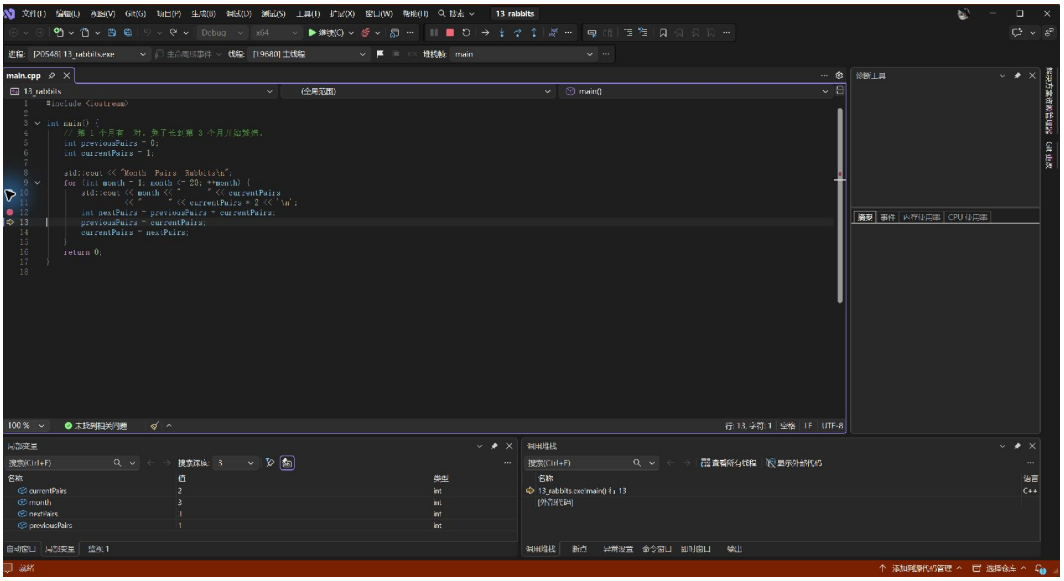


图 89 第 3 月计算下一月对数后的局部变量

month=3 时，当前月为 2 对，前一月为 1 对；执行赋值后 nextPairs=3，准备供下一月使用。

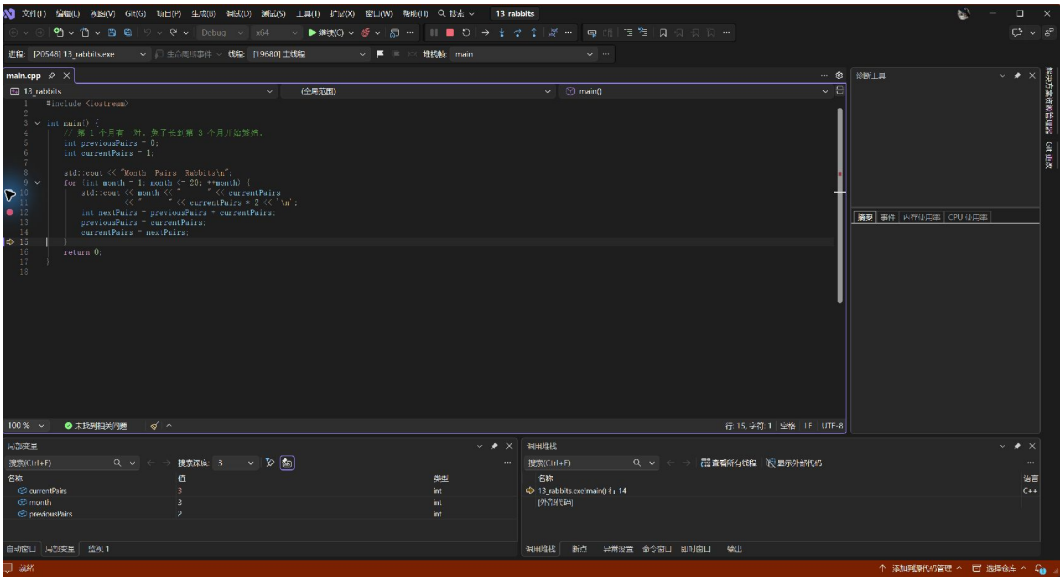


图 90　第 3 月循环末尾：为第 4 月准备状态

month 仍为 3，currentPairs 已更新为 3、previousPairs 更新为 2，供第 4 月迭代使用。

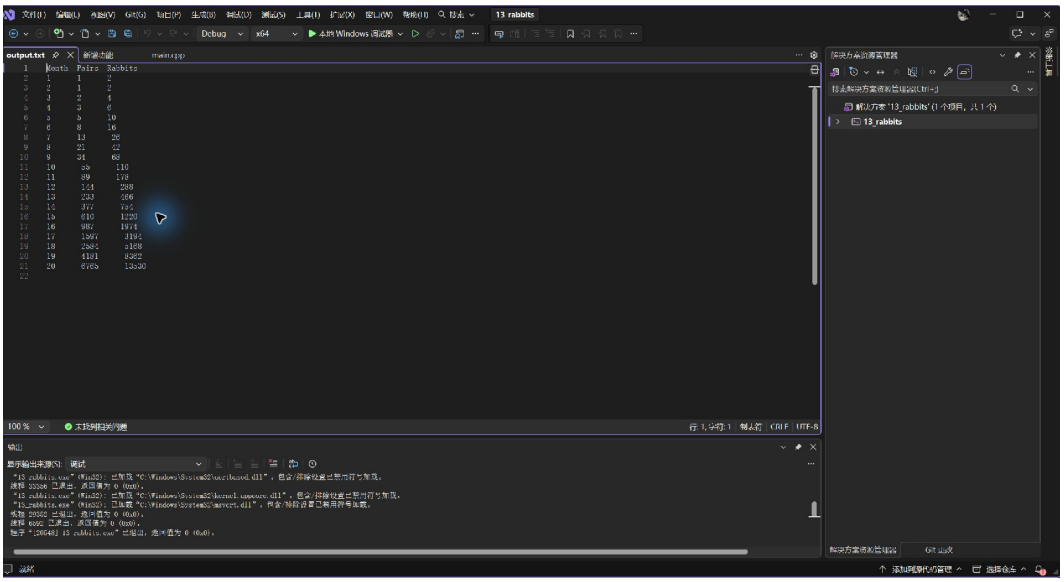


图 91　VS 运行生成的兔子 20 个月标准输出文件

output.txt 完整显示第 1 至 20 个月，前三个月为 2、2、4 只，第 20 个月为 6765 对、13530 只。Pairs 列为对数，Rabbits 列为总只数。

第 3 月输出 2 对 4 只，计算 nextPairs 后得到 3，此时 previousPairs=1、currentPairs=2。

两次替换后 month 仍为 3，currentPairs=3、previousPairs=2，为第 4 月准备好了状态。

20 个月的结果完整输出，第 20 个月为 6765 对、13530 只。

四、实验总结（本次实验的收获、体会、经验和教训）

在调用行按 F11 会进入 fact，按 F10 则执行完整调用后回到 main。第一次累乘后 result=5、val=4，返回时 result=120；形参和函数内的局部变量随该次调用结束而离开作用域，main 接收到的是返回值。

交换题用 temp 先保存 7，再改写两个引用；辗转相除先求出余数 12，再将 a、b 更新为 18、12。两题都需要先保存旧值再更新，赋值顺序直接影响结果。兔子题则先输出当前月，再准备下一月：month 仍为 3 时，更新后的 currentPairs=3 已经是第 4 月要用的对数，第 3 月输出仍为 2 对 4 只。

重载要结合实参和函数签名检查。三个 abs 进入不同类型版本，两个 Add 按参数个数区分；默认参数题在同一断点得到 num=100/result=5050 和 num=10/result=55。fact(3.14)也说明，编译警告和编译错误需要分开看：这次转换有 C4244 警告，程序仍能生成并输出 6。

递归需要同时检查终止条件和调用层次。斐波那契输入 6 时，n=3 的暂停点已有多层调用，n=2 进入 return 1；求幂中将 a 转换为 double 后进入实数重载，下一层指数从 3 变为 2。调试窗口中未完成赋值的变量暂不作结果，等赋值执行后再核对。

正常样例之外，还检查了除零、范围越界、质数边界、负数与零、负指数和整数溢出。36 项运行检查均通过，兔子的 20 个月结果也核对一致，第 20 个月为 13530 只。gcd(0,0)和 0^0 的输出采用本程序的演示约定，在解释结果时需要说明。